

Claude Code & AI 활용 교육 자료

8개 핵심 자료를 한 권으로 — 웰페리온 셋팅 레퍼런스

- ① Claude Code 기초
- ② 토큰 절약 3부작 (초·중·고급)
- ③ Claude Code 심화 (MCP·플러그인)
- ④ 주변 도구·개념 (AI 학습·Replit)

수록 자료

① Claude Code 기초

Claude Code 핵심 가이드

p. 4-15

설치부터 Plan Mode · Skill · MCP · Agent Team까지

② 토큰 절약 3부작

토큰 절약 — 초급편

p. 16-39

설정·습관 19가지, 개발자 아니어도 즉시 적용

토큰 절약 — 중급편

p. 40-64

CLAUDE.md 최적화 · MCP 관리 · Compaction 23가지

토큰 절약 — 고급편

p. 65-77

사전 인덱싱 · 3 에이전트 팀 · 혹 자동화 10가지

③ Claude Code 심화

MCP→CLI 전환 가이드

p. 78-89

토큰 17배 절감 · 성공률 100% 전환법

everything-claude-code 플러그인

p. 90-104

플러그인으로 Claude Code 비용 80% 절감

④ 주변 도구·개념

AI 학습 프레임워크

p. 105-117

쏟아지는 AI 용어·개념을 빠르게 익히는 법

Replit 입문 가이드

p. 118-126

코딩 없이 말로 만드는 홈페이지와 앱

이 자료로 무엇을 셋팅하나

8개 자료에서 welperion-automation 프로젝트에 바로 적용할 5가지를 뽑았습니다.

1 메인 지침서(CLAUDE.md) 신설

welperion-automation 루트엔 아직 지침서가 없습니다. 토큰 절약 중급편의 'CLAUDE.md 최적화' 항목을 기준으로 프로젝트 지침서를 새로 작성하세요.

2 MCP→CLI 전환 검토

guide(1)의 전환법으로 토큰을 크게 절감할 수 있습니다. telegram_bot이 이미 claude CLI를 호출 중이라 적용 여지가 큼니다.

3 토큰 절약 52팁 단계 적용

초·중·고급 3부작의 설정·습관을 settings.local.json과 평소 작업 방식에 단계적으로 반영하세요.

4 플러그인 도입 검토

everything-claude-code 플러그인으로 반복 작업의 비용을 줄일 수 있는지 검토하세요.

5 에이전트 팀 구조 정렬

핵심 가이드·고급편의 Sub Agent·Agent Team 개념을 notion-c-level-agents의 7 C-레벨 구조와 대조해 보강하세요.

Claude Code

핵심 가이드

설치부터 Agent Team까지, AI 코딩 도구의 모든 것

2026년 2월 | Opus 4.6 기준

프로젝트 브레인 • Plan Mode • Skill • MCP • Sub Agent • Agent Team

목차

Part 1. Claude Code 시작하기

- › Claude Code란?
- › 구독 플랜 및 설치
- › 개발 환경 (IDE) 설정
- › 토큰과 컨텍스트 윈도우
- › 프로젝트 브레인 (CLAUDE.md)
- › CLAUDE.md 작성 실전 팁
- › CLAUDE.md 레이어 구조
- › Rules 폴더와 Auto Memory

Part 2. 실전 개발 워크플로우

- › Task-Do-Verify 루프
- › Context Rot 방지하기
- › 병렬 작업과 Hook 시스템
- › 4가지 권한 모드
- › Plan Mode 활용법
- › 컨텍스트 관리 전략
- › 주요 슬래시 명령어

Part 3. 자동화와 확장

- › Skill 시스템
- › MCP (Model Context Protocol)
- › Sub Agent (서브 에이전트)
- › Agent Team
- › Git Worktree
- › 전체 워크플로우 조합

부록

- › 주요 키보드 단축키
- › 액션 아이템 3가지
- › 보안 주의사항

PART 1

Claude Code 시작하기

설치, 개발 환경 설정, 그리고 AI의 성능을 결정짓는 프로젝트 브레인까지

Claude Code란?

Claude Code는 Anthropic이 만든 에이전틱(Agentic) 코딩 도구입니다. 터미널에서 동작하며 코드베이스를 읽고, 파일을 편집하고, 명령어를 실행하고, 개발 도구와 통합됩니다. 자연어로 기능 구현, 버그 수정, 테스트 작성을 요청할 수 있고, Git 커밋/브랜치 생성/PR 오픈까지 자동화할 수 있습니다.

터미널, IDE(VS Code, JetBrains), 데스크톱 앱, 웹 브라우저, 모바일(iOS) 등 다양한 환경에서 사용 가능합니다.

구독 플랜 및 설치

플랜	가격	주요 특징
Pro	\$20/월	모든 기능 사용 가능, 사용량 제한 있음
Max 5x	\$100/월	Pro의 5배 사용량, Opus 4.6 접근, Agent Teams
Max 20x	\$200/월	Pro의 20배 사용량, 최대 우선순위
API	종량제	Sonnet 4.5: 입력 \$3/1M, 출력 \$15/1M 토큰

설치 명령어

```
# macOS / Linux / WSL
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# Homebrew
brew install --cask claude-code
```

Tip: 네이티브 설치에는 Node.js가 필요 없으며, 자동 업데이트가 내장되어 있습니다. 설치 후 프로젝트 폴더에서 `claude` 명령어로 바로 시작할 수 있습니다.

개발 환경 (IDE) 설정

Claude Code는 터미널에서 직접 쓸 수도 있지만, **VS Code**에서 사용하는 것이 가장 편합니다. VS Code에 Anthropic 공식 Claude Code 확장 프로그램을 설치하면 됩니다.

주의: 반드시 **Anthropic 공식** 확장 프로그램을 설치하세요. 비슷한 이름의 비공식 확장에는 악성 코드가 포함되어 있을 수 있습니다.

IDE 화면 구성

영역	위치	역할
파일 탐색기	왼쪽	프로젝트의 모든 파일/폴더 탐색
코드 편집기	가운데	파일 내용 확인 및 직접 수정
Claude Code 채팅	오른쪽/아래	AI에게 명령 전달, 결과 확인

Claude Code는 현재 열려 있는 파일을 자동으로 인식합니다. 특정 파일에 대해 작업을 시키려면 해당 파일을 편집기에서 열어둔 상태로 대화하세요.

토큰과 컨텍스트 윈도우

토큰 (Token)

AI가 텍스트를 처리하는 기본 단위. 대략 한 단어가 한 토큰 정도입니다. 자동 차에 비유하면 연료에 해당합니다.

컨텍스트 윈도우

AI가 한 번에 기억할 수 있는 대화의 양. Opus 4.6 기준 **200,000 토큰**(약 150,000 단어). 자동차의 연료 탱크에 해당합니다.

핵심: 이 탱크를 얼마나 효율적으로 쓰느냐가 Claude Code 활용의 핵심입니다.

프로젝트 브레인 (CLAUDE.md)

CLAUDE.md 는 프로젝트 폴더 최상위에 위치하는 파일로, Claude Code가 대화를 시작할 때 **가장 먼저 읽는 작업 지침서**입니다. 기술 스택, 코드 스타일, 프로젝트 구조 등의 정보를 담습니다.

비유: CLAUDE.md는 **배의 방향타**입니다. 출발할 때 각도를 1도만 틀어도 수천 킬로미터를 향해하면 도착지가 완전히 달라집니다. AI와 대화를 시작하기 전에 이 파일이 먼저 주입되므로, 여기에 뭘 적느냐에 따라 전체 프로젝트의 방향이 결정됩니다.

CLAUDE.md 생성 방법

- 1 `/init` 명령어로 자동 생성 (프로젝트 구조, 기술 스택, 아키텍처 자동 분석)
- 2 직접 규칙 추가 (테스트 코드 작성, 함수 이름 규칙, 에러 처리 패턴 등)
- 3 **200~500줄** 사이로 유지 (너무 길면 AI가 중간 내용을 놓침)

CLAUDE.md 작성 실전 팁

#	팁	이유
1	가장 중요한 규칙은 파일 맨 위에 배치	AI 모델의 Primacy Bias - 시작/끝 부분을 가장 잘 기억
2	API 문서 전체를 복사하지 말 것	토큰 낭비. 핵심 정보만 간결하게
3	같은 실수 2~3번 반복 시 규칙 추가	실제 문제가 생길 때 하나씩 추가하는 게 효과적
4	주기적으로 불필요한 규칙 정리	CLAUDE.md도 기술 부채가 쌓일 수 있음

CLAUDE.md 3가지 레이어

레이어	위치	적용 범위
글로벌	~/ .claude/CLAUDE.md	모든 프로젝트 공통 (개인 스타일, 코딩 규칙)
프로젝트	프로젝트 루트 CLAUDE.md	해당 프로젝트만 적용
관리자	기업용 전용	조직 전체 정책 강제

세 레이어가 병합되어 최종 지침이 완성됩니다. 프로젝트별로 다른 설정이 필요하면 프로젝트 레이어에서 덮어씁니다.

Rules 폴더와 Auto Memory

Rules 폴더

.claude/rules/ 폴더에 규칙을 주제별로 분리 저장할 수 있습니다. 설정 파일을 모듈화하는 것과 같은 개념입니다.

```
.claude/
rules/
  workflow.md      # 워크플로우 규칙
  design-rules.md  # 디자인 규칙
  technical-defaults.md # 기술 기본 설정
```

Auto Memory

MEMORY.md 파일에 중요한 정보를 자동 저장하는 기능입니다. 세션이 끝나도 다음 대화에서 이 정보를 기억합니다.

Part 1 핵심 요약: Claude Code의 성능은 코드를 입력하기 전에 이미 결정됩니다. CLAUDE.md, Rules 폴더, 메모리를 제대로 설정하면 AI가 프로젝트를 정확히 이해한 상태에서 출발합니다. 설정에 5분 투자하면 이후 수십 시간을 아낄 수 있습니다.

PART 2

실전 개발 워크플로우

검증 루프, Plan Mode, 컨텍스트 관리로 AI 코딩의 품질과 속도를 모두 잡는 법

Task-Do-Verify 루프

AI 코딩의 핵심 워크플로우입니다. 대부분의 사람들이 AI 코딩에 실망하는 이유는 바로 **검증 단계를 빼먹기** 때문입니다.

Task (작업 지시)



Do (AI가 실행)



Verify (결과 검증)



반복

디자인 작업

- 원하는 디자인 스크린샷 전달
- 결과물을 다시 스크린샷으로 찍기
- 원본과 비교하여 수정 지시
- 서너 번 반복으로 완성

코딩 작업

- 테스트 코드 자동 작성/실행 지시
- 테스트 통과 시 다음 단계
- 실패 시 AI가 자동 수정
- TDD(테스트 주도 개발) 방식

AI의 진짜 가치: 완벽함이 아닌 속도에 있습니다. 80% 완성도에서 시작해서 빠르게 수렴하는 것이 핵심입니다.

Context Rot 방지하기

AI 코딩에서 가장 흔한 실패 패턴: 처음에 잘 되다가 30분 후 갑자기 아무것도 동작하지 않는 현상.

Context Rot이란? 오류가 누적되면서 AI가 자기가 이전에 뭘 했는지 혼란스러워하는 현상입니다. 매 단계마다 검증하는 습관이 이 문제를 근본적으로 방지합니다. **귀찮아도 매번 검증하는 습관이 결국 가장 빠른 길입니다.**

병렬 작업과 Hook 시스템

병렬 작업

IDE에서 탭을 여러 개 열어 각각 다른 작업을 시킬 수 있습니다. 추천 동시 실행 수는 **3~4개**입니다.

Hook 시스템

Claude Code가 특정 동작을 할 때 자동으로 실행되는 사용자 정의 스크립트입니다.

Hook 이벤트

발생 시점

활용 예시

PreToolUse

도구 실행 전

보호 파일 편집 차단

PostToolUse	도구 실행 후	자동 코드 포매팅
Stop	응답 완료 시	완료 알림 소리
Notification	알림 전송 시	macOS 알림
SessionStart	세션 시작 시	환경 초기화
UserPromptSubmit	프롬프트 제출 시	입력 검증

Hook 종료 코드

코드	동작
Exit 0	작업 진행 허용
Exit 2	작업 차단 (stderr 메시지가 Claude에 피드백)
기타	작업 진행, stderr는 로그에만 기록

4가지 권한 모드

모드	설명	추천 대상
기본 모드	파일 변경마다 허락 구함	안전 최우선
자동 수정 모드	기존 파일 수정 자동, 새 파일만 확인	초보자 추천
Bypass Permissions	모든 권한 AI에게 위임	격리 환경만
Plan Mode	설계를 먼저, 코드 수정 불가	복잡한 작업

Bypass Permissions 주의: AI가 시스템 파일을 통째로 날려버린 사례가 보고된 적 있습니다. 격리된 환경에서만 사용하세요.

Plan Mode - 가장 강력한 기능

짓기 전에 설계도를 먼저 그리는 방식. Plan Mode에서 Claude Code는 코드를 직접 수정하지 않습니다.

- 1 프로젝트 구조를 읽고 분석합니다
- 2 질문이 있으면 사용자에게 물어봅니다
- 3 어떻게 구현할지 상세한 계획을 세웁니다
- 4 사용자가 검토하고 승인하면 코드를 작성합니다

효과: 35분 걸리는 작업이 15분으로 단축. 계획 1분 투자 = 빌드 10분 절약. 특히 데이터베이스, 결제, 인증 같은 복잡한 기능에서 효과 극대화.

비유: 플랜 없이 코딩하는 건 네비게이션 없이 처음 가는 곳을 운전하는 것과 같습니다. 도착은 하겠지만, 기름값이 3배로 나옵니다.

컨텍스트 관리 전략

첫 메시지를 입력하기도 전에 이미 **25% 이상**이 소비되어 있습니다 (시스템 프롬프트, 도구 설명, MCP, 메모리 등).

#	전략	설명
1	<code>/context</code>	현재 컨텍스트 사용량 확인
2	<code>/compact</code>	대화를 고밀도 요약으로 압축
3	짧고 구체적인 프롬프트	핵심만 전달하여 토큰 절약
4	Sub Agent 활용	리서치를 별도 에이전트에 위임, 메인 컨텍스트 보호
5	Extended Thinking	사고 과정 토큰이 컨텍스트에 누적되지 않음
6	모델 적재적소	단순 작업 Sonnet, 복잡한 작업 Opus

주요 슬래시 명령어

명령어	설명
<code>/init</code>	CLAUDE.md 자동 생성
<code>/context</code>	컨텍스트 사용량 확인
<code>/compact</code>	대화 압축
<code>/clear</code>	대화 완전 초기화
<code>/cost</code>	토큰 비용 확인
<code>/model</code>	모델 변경 (Sonnet / Opus / Haiku)
<code>/review</code>	최근 변경사항 코드 리뷰
<code>/config</code>	설정 인터페이스
<code>/permissions</code>	권한 규칙 관리
<code>/hooks</code>	Hook 설정
<code>/agents</code>	서브에이전트 관리
<code>/add-dir</code>	추가 디렉토리 포함

PART 3

자동화와 확장

Skill, MCP, Sub Agent, Agent Team으로 개인이 팀 수준의 생산성을 내는 법

Skill 시스템

Skill은 AI를 위한 업무 매뉴얼입니다. 반복 작업을 단계별 체크리스트로 파일에 저장합니다. /스킬이름 으로 바로 실행할 수 있습니다.

Skill 구조

```
# 저장 위치
.claude/skills/my-skill.skill.md

# 파일 형식 (Markdown)
---
name: my-skill
description: 이 스킬의 간단한 설명
---
실제 작업 단계가 여기에 적힙니다...
```

영리한 설계: Front Matter(이름/설명)만 먼저 로드하므로 약 60~70 토큰만 소비. 실제 내용은 호출 시에만 로드. **Skill을 100개 만들어도** 컨텍스트를 거의 차지하지 않습니다.

Skill 활용 사례

Skill	수동 작업	Skill 적용
리드 스크래핑	30분+	90초
이메일 분류/답장	1시간+	수 분
주제별 리서치	수 시간	수 분
맞춤 웹사이트 생성	수 일	30초

Skill 만드는 방법

- 1 작업을 음성이나 텍스트로 Claude에게 설명
- 2 Claude가 Skill 파일 형식으로 자동 정리
- 3 70% 정확도에서 시작 (괜찮습니다!)
- 4 테스트하며 실패 부분 수정 (2~3회 반복)
- 5 98~99% 정확도 달성

Tip: 새로 만든 Skill은 반드시 새로운 세션에서 테스트하세요. 기존 대화 컨텍스트가 결과에 영향을 줄 수 있습니다.

MCP (Model Context Protocol)

MCP는 다른 사람이 만들어 놓은 Skill과 비슷합니다. Gmail, Slack, Notion, Google Sheets 등 외부 서비스와 Claude Code를 연결하는 플러그인입니다.

MCP vs Skill 비교

항목	MCP	Skill
토큰 사용	수천 토큰	60 토큰
처리 속도	이메일당 수 초	0.3초
용도	프로토타입용	운영용

추천 워크플로우: MCP로 빠르게 기능 테스트 → 잘 작동하면 API 직접 호출하는 Skill로 변환 → 토큰 사용량 극적 감소

최추천 MCP: Anthropic Chrome DevTools MCP. Chrome 브라우저를 직접 제어하여 웹 페이지 열기, 버튼 클릭, 데이터 추출이 가능합니다. API가 없는 서비스에서 데이터를 가져올 때 필수입니다.

Sub Agent (서브 에이전트)

메인 Claude Code가 특정 업무를 별도 에이전트에게 위임하는 개념입니다. Sub Agent는 자기만의 별도 컨텍스트에서 작업하고, 결과 요약만 메인에게 돌려줍니다.

내장 Sub Agent

에이전트	모델	용도
Explore	Haiku (빠름/저렴)	코드베이스 탐색, 파일 검색
Plan	상속	Plan Mode에서 리서치
General-purpose	상속	복잡한 멀티 스텝 작업
Bash	상속	별도 컨텍스트에서 명령어 실행

유용한 커스텀 Sub Agent

에이전트	역할
리서치 에이전트	저렴한 Sonnet으로 방대한 자료 검색/요약
코드 리뷰 에이전트	새로운 시각에서 코드 품질 검토
품질 보증 에이전트	테스트 코드 자동 작성/실행

확률의 수학: 각 Sub Agent 성공률 95%일 때, 10개를 동시에 돌리면 전체 성공률은 **59%**입니다 (0.95의 10제곱). 작업을 잘게 쪼개서 단

순한 일을 시키는 것이 핵심입니다.

Agent Team

Sub Agent의 진화형. 팀원들이 **서로 소통**하며 자율적으로 협업합니다.

Agent Team 특징

특징	설명
독립 인스턴스	각 팀원이 자기만의 컨텍스트, CLAUDE.md, MCP 접근 권한 보유
자율 협업	공유 태스크 보드에서 진행 상황 확인하며 자율 작업
적대적 토론	두 에이전트가 반대 입장에서 토론하여 품질 향상 (GAN 원리)
활성화	<code>CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1</code> 설정 필요

비용 주의: 일반 세션 대비 **7배** 토큰 소비. 에이전트 10명 x 15분 = 약 8만원. 대규모 작업(보안 감사, 코드 리팩토링)에만 사용 권장.

Git Worktree

Agent Team과 함께 사용할 때 효과적입니다. 각 Branch가 별도 작업 디렉토리를 가져서 **파일 충돌 없이 병렬 개발**이 가능합니다. 한 에이전트는 소개 페이지, 다른 에이전트는 연락처 페이지, 또 다른 에이전트는 서비스 페이지를 동시에 만들고 나중에 합칩니다.

전체 워크플로우 조합



반복 작업은 **Skill**로, 외부 연동은 **MCP/API Skill**로 자동화합니다. 이 워크플로우를 한번 구축해 놓으면 어떤 프로젝트든 동일한 품질 수준으로 빠르게 진행할 수 있습니다.

부록: 주요 키보드 단축키

단축키	설명
Escape	Claude 중지 (Ctrl+C가 아님!)
Escape x 2	이전 메시지 목록으로 점프
Shift+Tab	Plan Mode 전환
Cmd+T	Extended Thinking 토글
Cmd+P	모델 선택기
Ctrl+G	외부 에디터에서 편집

Ctrl+O

상세 모드(verbose) 토글

Ctrl+B

현재 작업을 백그라운드로 전환

부록: 오늘 당장 실행할 액션 아이템

- 1 CLAUDE.md 만들기:** `/init` 으로 시작. 자주 하는 실수나 반드시 따라야 할 규칙을 3줄만 추가해 보세요.
- 2 Plan Mode 사용하기:** 다음 작업에서 바로 빌드하고 싶은 충동을 참고, 계획을 먼저 세워보세요.
- 3 Skill 하나 만들기:** 매일 반복하는 작업 하나를 Skill로 만들어 보세요. 70% 정확도로 시작해서 점점 개선하면 됩니다.

부록: 보안 주의사항

반드시 기억하세요:

AI가 작성한 코드에는 보안 취약점이 숨어 있을 수 있습니다. 돈을 받고 서비스를 운영하기 전에는 반드시 보안 검토를 하셔야 합니다.

특히 **결제 기능**이나 **개인정보**를 다루는 서비스라면 전문가의 보안 리뷰를 꼭 받으세요.

Claude Code는 엄청난 도구이지만, 모든 도구가 그렇듯 **제대로 알고 쓸 때 진가를 발휘합니다.**

Claude Code 핵심 가이드 | 2026년 2월 기준 | Opus 4.6

AI 도구는 빠르게 발전하고 있습니다. 최신 정보는 공식 문서를 참고하세요.

CLAUDE CODE · 토큰 절약 시리즈 · 초급편

토큰 소모 걱정, 이제 끝냅시다

공식 문서 · 기술 블로그 · 커뮤니티를 전부 뒤져서 모은 52가지 팁 —
오늘은 **초급 19가지**를 전부 설명합니다

”

토큰을 모두 소모하였습니다

돈 내고 쓰는데 한 시간도 안 됐는데 이 메시지가 떠요.

근데 이거 클로드 탭도, 플랜 탭도 아니에요.

쓰는 방식이 문제입니다.

전체 구성

총 52가지 팁 — 난이도별로 쪼갬어요

난이도	개수	다루는 내용	상태
초급	19가지	설정 하나, 습관 하나. 개발자 아니어도 즉시 적용	지금 공개
중급	23가지	CLAUDE.md 최적화, MCP 관리, 컴팩션	다음 편
고급	10가지	서브에이전트 구조, 프로젝트 최적화, 자동화	추후 공개

핵심 개념

클로드는 **매번 전체 대화를 재로딩**해요

- 1 카카오톡 단톡방은 새 메시지만 읽음. 클로드는 메시지를 보낼 때마다 **1번 메시지부터 전부 다시 읽음**
- 2 메시지당 평균 500 토큰 가정 → **10번째 메시지 = 27,500 토큰** (5,000이 아님)
- 3 **30번째 메시지 = 232,000 토큰** — 첫 번째 메시지보다 **31배** 비쌈
- 4 해결책은 단순: 작업 끝나면 `/clear`로 초기화 → 다시 0부터 시작

1

새 작업 시작 전엔 반드시 **/clear**

기능 구현이 끝났으면 다음 작업 시작 전에 무조건 **/clear** 해주세요.
토큰을 절약하는 가장 쉽고 빠른 방법입니다.

500

1번째 메시지 토큰

27,500

10번째 메시지 토큰

232,000

30번째 메시지 토큰

2

프롬프트는 범위를 제한해서 보내기

✗ 비효율

"이 코드 개선해줘"

→ 클로드가 프로젝트 전체를 뒤집음. 파일 이것저것 열고 탐색하는 과정에서 토큰 낭비

✓ 효율

"auth.js의 verifyUser 함수에 입력값 검증 추가해줘"

→ 그 파일 하나만 열고 바로 작업. **최대 10배 차이**

핵심은 파일명, 함수명, 조건을 명시적으로 주는 것. 클로드가 알아서 탐색하게 두지 마세요.

3

간단한 명령은 묶어서 한 번에 보내기

- "요약해줘" → "포인트 뽑아줘" → "제목 추천해줘" 따로 3번 = **3배 과금**
- "요약하고, 핵심 포인트 뽑고, 제목 추천해줘" 한 번에 = **1배 과금 + 품질 향상**
- 클로드가 잘못 답했을 때 **"아니 그게 아니라..."** 후속 메시지 금지
- 대신 **원본 메시지의 편집 버튼** 으로 수정하고 재생성 → 잘못된 기록이 컨텍스트에서 사라짐

4

붙여넣기는 꼭 필요한 것만

파일이나 문서를 통째로 붙여넣기 전에 한 번만 물어보세요.

"클로드가 이걸 전부 읽어야 하나?"

- 버그가 함수 하나에 있다면 → **그 함수만** 붙여넣기
- 단락 하나의 맥락만 필요하다면 → **그것만** 넣기
- 파일 전체를 던져주면 → **전체가 다 토큰**

5

작업할 때 **자리 뜨지 마세요**

Shift+Tab 자동 수락 모드로 켜놓고 자리 비우는 분들 많으신데, 생각보다 위험해요.

- 클로드가 **잘못된 방향** 으로 달려가는 경우 있음
- 같은 파일을 반복해서 읽는 **루프에 빠지는 경우** 있음
- 초반에 발견해서 멈추면 **수천 개의 토큰** 을 아낄 수 있음
- 작업 초반만이라도 **옆에서 지켜보면서** 의도치 않은 루프 확인 권장

6

기본 모델을 소넷으로 고정하세요

처음 켜올 때 기본 모델이 오퍼스로 잡혀 있는 경우 있어요. 이 상태로 계속 작업하면?

오퍼스는 소넷 대비 4~5배 더 토큰을 소모해요.

설정 파일에 한 줄만 추가해서 소넷으로 고정하세요.

필요할 때만 `/model opus` 또는 `/model sonnet`으로 전환.

7

작업에 맞는 모델 선택하기

모델	비유	언제 쓰나	비용
Haiku	주니어 개발자	학습, 간단한 실습, 단순 질문	가장 저렴
Sonnet	중급 개발자	일반 코딩, 기능 구현, 파일 수정	기본값 권장
Opus	시니어 개발자	복잡한 아키텍처 설계, 깊은 디버깅	소넷의 4~5배

시니어한테 복사 붙여넣기 시키는 건 낭비잖아요. 모델 선택만으로 **50~70% 절감** 가능.

8

안 쓰는 기능은 꺼두세요

- 웹 검색, 커넥터, 확장 사고 기능이 **켜져 있기만 해도** 매 응답마다 추가 토큰 소모
- 특히 **Extended Thinking** (확장된 사고) 기능 — 기본으로 활성화되어 있는 경우 있음
- 이 기능이 최대 **3만 토큰** 을 출력 토큰으로 과금. 단순 파일 수정인데도요.
- 클로드가 3만 토큰을 **생각하는 데** 쓰고 있을 수 있다는 것. 필요할 때만 켜세요.

9

Memory와 사용자 설정 저장해두기

새 채팅 열 때마다 아래 같은 걸 입력하시나요?

"나는 백엔드 개발자야. 코틀린과 스프링 위주로 설명해줘. 코드는 간결하게 작성해줘."

- 이게 매번 3~5개 메시지 를 잡아먹어요
- Settings → Memory and User Settings 에 역할, 스타일을 한 번만 저장
- 클로드가 모든 새 채팅에 자동으로 적용 . 매번 입력할 필요 없음

10

픽크 타임을 피해서 작업하세요

픽크 시간대

- 미국 동부 오전 8시 ~ 오후 2시
- 한국 시간 **밤 10시 ~ 새벽 4시**
- 같은 작업을 해도 토큰이 더 빠르게 소진

한국 분들은 유리해요

- 한국 업무 시간(9시~6시)은 **미국 비픽크** 시간대
- 평소대로 낮에 작업하면 됨
- 대규모 리팩토링은 **주말에** 하면 더 넉넉

11

리셋 타이밍을 전략적으로 활용하세요

상황 A

리셋이 곧 다가오는데 할당량이 아직 많이 남아 있다면?

과감하게 써버리세요. 어차피 리셋되면 초기화.

상황 B

할당량이 얼마 안 남았는데 리셋까지 시간이 많이 남았다면?

잠깐 쉬세요. 풀 예산으로 다시 시작하는 게 낫습니다.

12

하루를 2~3 세션으로 나눠서 사용하세요

클로드는 자정 리셋이 아니에요. 최근 5시간 동안 사용한 양만 한도에 포함되는 롤링 윈도우 방식.



오전 세션 — 핵심 작업 집중 처리. 한도 전부 써도 OK



오후 세션 — 오전(5시간 전) 사용량이 윈도우에서 빠짐. 새 한도로 재시작



저녁 세션 — 오후 사용량도 만료. 같은 요금제인데 **체감 한도 훨씬 넉넉**

13

1세션 = 1작업 원칙

버그 3개 고치면서 기능 2개 추가하고 리팩토링까지 한 세션에 다 하려고 하면?

- 컨텍스트가 섞이면서 **오염** 됨
- 클로드가 이전 작업 내용을 계속 들고 다니면서 **토큰 낭비**
- 심하면 헛갈려서 **할루시네이션** 발생
- 원칙: **1세션 = 1논리 작업** . 끝나면 `/clear` 하고 다음 작업 시작

14

세션 시작 전에 **작업 목록 미리 정리**하기

클로드 코드 켜고 바로 시작하시나요? 세션 시작 전에 딱 3가지만 해두세요.

- 1 오늘 처리할 작업 목록 미리 정리
- 2 우선순위 정해서 가장 중요한 것부터 시작
- 3 5시간 세션을 **하나의 스프린트**처럼 대하기 → 집중력 유지 + 불필요한 탐색 토큰 절약

15

Extra Usage 안전장치 켜두기

토큰 절약 팁은 아니지만, 중요한 작업 중에 갑자기 막히는 상황을 방지하는 안전장치예요.

- Pro, Max 구독자 → **Settings** → **Usage** → **Overage 기능** 켜기
- 세션 한도 도달 후에도 **API 요금으로 과금** 되는 방식으로 계속 진행
- **월별 지출 한도** 설정 가능 → 예상치 못한 과금 방지

16

/rename과 /resume 활용하기

- `/clear` 로 컨텍스트 비우기 전에 `/rename` 으로 세션 이름 지정
- 나중에 `/resume` 명령어로 그 대화로 쉽게 돌아갈 수 있음
- `/clear` 했다고 완전히 없어지는 게 아님 — 이름 붙여서 보관 가능
- 작업 히스토리가 필요하거나 나중에 참고해야 할 대화에 유용

17

`/context` · `/cost` · `/usage` 활용하기

- `/ctx` `/context` — 토큰을 어디서 얼마나 쓰는지 컴포넌트별로 확인 (대화 기록, MCP 오버헤드, 로드된 파일)
- `/cst` `/cost` — 현재 세션의 실제 토큰 사용량 + 예상 비용
- `/use` `/usage` — 커런트 세션 + 주간 사용량 한눈에 확인

새 세션에서 아무것도 안 하고 `/context` 실행해봤더니 시스템 프롬프트·도구 정의·메모리 파일 때문에 이미 51,000 토큰이 쓰여 있었어요. MCP 없어도요.

18

사용량 대시보드 옆에 열어두기

- 클로드 계정 대시보드를 작업하는 동안 옆 탭에 열어두기. 20~40분마다 확인
- 사용량 한도에 갑자기 부딪히는 상황을 미리 방지
- 자동화 버전: 30분마다 사용량 확인해서 슬랙·문자로 알림 보내는 스크립트 작성
- `npm install -g @ryoppippi/ccusage` → CLI에서 일별/월별/실시간 사용량 확인 가능

19

플랜에 따라 전략을 다르게 가져가세요

플랜	소넷 주간	오피스 주간	5시간당	전략 핵심
Pro \$20	40~80시간	매우 제한적	약 45개	소넷 고정 필수
Max \$100	140~280시간	15~35시간	약 225개	오피스 적당히 + 모니터링
Max \$200	240~480시간	20~40시간	약 900개	자유롭게, 낭비는 금지

오피스를 기본 모델로 쓰다가 2일 만에 주간 사용량 80%가 찼다는 분들도 있어요. Pro 플랜은 소넷 고정이 핵심.

다음 편 예고

초급 19가지, 어렵거나 복잡한 거 없었죠? 다음 편은 **중급 23가지**입니다

CLAUDE.md 최적화 · MCP 관리 · 컴팩션

구독과 알림 설정으로 업로드되자마자 챙겨보세요.

초급 ✓ 완료

중급 → 다음 편

고급 → 추후 공개

전체 구성

총 52가지 팁 — 중급편은 여기예요

난이도	개수	다루는 내용	상태
초급	19가지	설정 하나, 습관 하나. 개발자 아니어도 즉시 적용	공개 완료
중급	23가지	CLAUDE.md 최적화, MCP 관리, Compaction	지금 공개
고급	10가지	사전 인덱싱, 3 에이전트 팀 구조, 혹은 자동화	추후 공개

.claudeignore 파일 만들기

클로드는 탐색 시 `node_modules`, `.next`, `dist` 같은 파일까지 전부 읽으려 해요.
큰 프로젝트면 수십 기가 — 토큰이 순식간에 녹습니다.

프로젝트 루트에 생성

```
node_modules/  
.next/  
dist/  
build/  
*.log  
*.lock  
__pycache__/  
coverage/
```

활용 팁

- **.gitignore**처럼 클로드가 읽지 않을 파일 지정
- "이 프로젝트 **.claudeignore** 추천해줘" 하면
기반으로 잘 만들어 줌
- 프로젝트 초반 한 번만 만들면 **계속 유효**

2

CLAUDE.md는 200줄 이하로 유지하기

CLAUDE.md는 세션 시작 때만 읽히는 게 아닙니다. 매 메시지마다 읽혀요.

5,000

장황한 CLAUDE.md 토큰

700

간결한 CLAUDE.md 토큰

4,300

메시지당 절약 토큰

- "이 프로젝트는..." 같은 서술형 문장 제거
- heading·list·table로 구조화
- 규칙은 최소 단위로 축약, 코드 예시 과도 포함 금지
- 200줄 넘기 시작하면 다이어트 타이밍

상세 지식은 **스킬로 분리하기**

CLAUDE.md를 200줄 이하로 유지하면서 나머지 내용은 어디에? **스킬 파일로 분리**하세요.

추천 구조

```
# 항상 로드
CLAUDE.md

# 필요할 때만 로드
docs/api-guide.md
docs/db-guide.md
docs/deploy-guide.md
```

왜 효과적인가

- CLAUDE.md: **항상** 로드 → 핵심 규칙만
- 스킬 파일: **필요할 때만** 메모리 로드
- 세션당 최대 **15,000 토큰 절약** 가능

4

CLAUDE.md에 5가지 절약 규칙 추가하기

직접 넣어두면 클로드가 스스로 낭비를 줄여요.

- 1 이미 읽은 파일은 다시 확인하지 않는다
- 2 불필요한 도구 호출은 하지 않는다
- 3 가능한 도구 호출은 동시에 실행한다
- 4 20줄 이상의 불필요한 출력은 서브에이전트에 위임한다
- 5 사용자가 이미 설명한 내용을 다시 반복하지 않는다 — 1·5번이 특히 중요

CLAUDE.md에 이미 결정된 내용 적어두기

한 번 저장해두면 다시는 프롬프트로 타이핑하지 않아도 됩니다.

- 안정적인 아키텍처 결정들 — 이미 정해진 방향
- 코딩 규칙과 컨벤션 — 매번 설명할 필요 없음
- 빌드·테스트·배포 명령어 — 자주 묻는 것들
- 자주 하는 실수와 해결책 — 반복 실수 방지

이 파일이 탄탄할수록 매번 타이핑해야 하는 프롬프트가 효과적으로 줄어듭니다.

단, 자동 학습 설정 시 파일이 너무 길어지지 않는지 주기적으로 확인하세요.

6

@로 파일 전체 참조 **지양**하기

@ 기호로 파일을 참조하면 해당 파일 내용 **전체**가 컨텍스트에 포함됩니다.

파일 크기	참조 방식
500줄 이하	전체 참조 가능
500 ~ 2,000줄	필요한 구간만 지정
2,000줄 이상	함수·클래스 단위로 분리해서 참조

✗ 비효율

"전체 레포 보고 버그 찾아봐"

✓ 효율

"auth.js의 verifyUser 함수 확인해줘"

7

60% 시점에 수동으로 압축하기

클로드는 컨텍스트 95% 차면 자동 압축(compaction)을 해요. 하지만 그때는 이미 컨텍스트 품질이 상당히 저하된 상태입니다.

1

`/context` 명령어로 사용률 확인 → 60% 도달 시 수동으로 대응

2

보존할 내용을 함께 지시하며 `/compact` 실행

3

압축을 3~4번 해야 할 정도로 길어지면 세션 요약 후 `/clear` 후 새로 시작

"/compact 실행해줘. 현재 구현 중인 로그인 기능의 핵심 로직과 발생한 에러 내역은 반드시 보존해줘."

8

자리 비우기 전에 압축하거나 클리어하기

화장실 다녀오거나 커피 한 잔 마시러 자리를 비우는 것, 토큰 소비에 영향을 줍니다.

5분

캐시 만료 시간

전부

5분 후 돌아오면 처음부터 재처리



나가기 전 `/compact` 또는 `/clear`

갑자기 사용량이 확 오르는 느낌이 드는 게 바로 **캐시 만료** 때문입니다.

자리 비울 거라면 반드시 `/compact` 나 `/clear` 먼저.

셸 명령어의 긴 출력 주의하기

클로드가 셸 명령어를 실행하면 전체 출력이 컨텍스트 윈도우로 그대로 들어옵니다.

위험한 패턴

- 커밋 200개짜리 `git log`
- verbose 옵션 켜 전체 테스트 결과
- 빌드 경고 메시지 가득한 로그

대응 방법

- "전체 테스트" → "auth 모듈 테스트만"
- 혹은 `head` / `tail` 로 자동 설정
출력을 상한선
- 필요 없는 명령어는 해당 프로젝트에서 권한 거부

10

Extended Thinking 기능 관리하기

눈에 보이지 않아서 더 무서운 토큰 낭비입니다.

30,000

Extended Thinking 최대 토큰

기본

활성화 상태로 시작

/config

설정 변경 방법

- 단순 파일 수정인데도 3만 토큰을 생각에 쓸 수 있음
- 복잡한 아키텍처 설계·깊은 디버깅 → **켜두기**
- 단순한 작업 → **꺼두기**
- **/config** 또는 환경변수로 thinking 예산 낮추기 가능

11

MCP 서버 최소화하기

MCP 서버를 연결하면 **매 메시지마다** 해당 서버의 모든 도구 정의가 컨텍스트에 로드됩니다.

MCP 서버	토큰 소비 (메시지당)
Playwright MCP	17,600 토큰
Supabase MCP 추가	+20,900 토큰
둘 다 연결 시	38,500 토큰/메시지

아무 말 안 하고 엔터만 쳐도 기본으로 38,500 토큰. `/mcp` 로 지금 바로 확인해보세요.

안 쓰는 서버 3개만 끊어도 체감이 확 달라집니다.

12

MCP tool search 옵션 설정하기

MCP 서버를 여러 개 사용하는 분들께 특히 유용한 팁입니다.

옵션 설명

- 기본값 **auto** : MCP 도구가 10%에만
컨텍스트의 초과 시 발동
- **true** 로 설정: 항상 tool search 활성화
- 여러 MCP 서버 + 컨텍스트 관리 고민 → **true 권장**

적용 대상

- MCP 서버를 3개 이상 연결한 경우
- 자주 컨텍스트 부족을 느끼는 경우
- 상세 설정은 공식 문서 참고

4단계로 MCP 서버 줄이기

- 1 전체 목록화 — `~/.claude/settings.json` 의 `mcpServers` 키 확인 후 전부 적기
- 2 도구 수 확인 — 각 서버에 어떤 도구가 있는지, 설명이 얼마나 장황한지 체크
- 3 토큰 비용 추정 — 예상 토큰 = $(\text{도구 수} \times 200) + (\text{설명 글자 수} \div 4)$
- 4 상위 오버헤드 서버 제거 — 비용 내림차순 정렬 시 2~3개가 전체 대부분 차지, **그것부터 정리**

14

글로벌 vs 프로젝트 레벨 MCP 분리하기

모든 MCP를 글로벌에 넣으면 가벼운 프로젝트에서도 전부 로드됩니다.

글로벌 설정 (~/.CLAUDE/SETTINGS.JSON)

Filesystem MCP 하나만

거의 모든 작업에 필요하니까

프로젝트 설정 (.CLAUDE/SETTINGS.JSON)

DB MCP → DB 프로젝트에만

Playwright MCP → 프론트 프로젝트에만

백엔드 프로젝트에서 Playwright가 로드되는 낭비가 사라집니다.

분산 메모리 구조 만들기

CLAUDE.md를 "데이터가 어디 있는지 알려주는 인덱스"로 활용하세요.

구조 예시

```
# 항상 로드  
CLAUDE.md  
  
# 필요시만  
docs/api-guide.md  
docs/db-guide.md  
docs/deploy-guide.md
```

3가지 장점

- 기본 컨텍스트가 훨씬 가벼워짐
- 기능별 분리로 유지보수 쉬워짐
- 필요한 정보를 더 빠르고 정확하게 찾음

CLAUDE.md에 명시: "API 관련 작업이 필요하다면 docs/api-guide.md를 읽어라."

16

대규모 작업 후 그냥 `/clear` 하지 않기

컨텍스트가 너무 커지면? 날리기 전에 **저장하고 이어받기**를 하세요.

- 1 **진행 상황 저장** — "지금까지 진행한 내용을 `progress.md` 로 정리해줘. 완료된 것, 진행 중, 다음 단계 포함해서."
- 2 **컨텍스트 초기화** — `/clear` 로 깨끗하게 리셋
- 3 **이어서 작업** — "@progress.md 읽고 이어서 작업 계속해줘."

Projects에 반복 파일 업로드하기

같은 PDF나 문서를 여러 채팅에서 반복 업로드하면 **매번 토큰으로 변환됩니다.**

❌ 반복 업로드

10번 업로드 = **10번 토큰 소모**

매 채팅마다 처음부터 처리

✅ Projects 활용

한 번 업로드 → **캐시 참조**

추가 토큰 소모 없음

claude.ai 웹의 Projects 기능 — 반복 참고 문서가 있다면 **반드시 활용하세요.**

18

플랜 모드 먼저 활용하기

토큰 낭비의 최대 원인 = 잘못된 방향으로 코드를 잔뜩 작성하고 전부 버리는 상황.

- 1 Shift+Tab 2번으로 플랜 모드 진입
- 2 클로드가 어떤 파일을 수정할지, 어떤 순서로 할지, 리스크는 뭔지 아웃라인 작성
- 3 확인하고 승인하면 그때 실행으로 넘어감

CLAUDE.md 추가 권장: "95% 확신이 생기기 전에는 어떤 변경도 하지 말아줘. 그 확신에 도달할 때까지 추가 질문을 해줘."

19

상태 표시줄 (Status Line) 설정하기

터미널 하단에 실시간으로 현황을 띄워두세요.

터미널 하단 표시 예시

현재 모델	컨텍스트 사용률	토큰 수
claude-sonnet-4-6	 62%	124,800 / 200,000

터미널에서 `/status line` 입력 후 설정 적용. 압축 타이밍을 잡기 훨씬 쉬워집니다.

ccusage로 사용량 정밀 추적하기

대시보드 수동 확인이 번거롭다면 ccusage를 설치하세요.

설치

```
npm install -g @ryoppippi/ccusage
```

주요 명령어

```
ccusage daily  
ccusage monthly  
ccusage blocks --live  
ccusage daily --breakdown
```

할 수 있는 것

- 오늘 토큰 소비량 + 비용 분석
- 5시간 빌링 윈도우 실시간 모니터링
- 모델별 비용 분석
- 날짜 범위 필터로 특정 기간 추적
- 갑자기 토큰 많이 나간 날 원인 추적

토큰 사용량 확인하는 3가지 방법

1

/context 명령어 — 컴포넌트별 토큰 사용량

시스템 프롬프트 · MCP 도구 · 메모리 파일 · 대화 메시지 각각 확인

2

로그 파일 직접 확인 — **~/.claude/projects/** 폴더의 JSONL

각 메시지의 **usage** 필드 → **input_tokens**, **output_tokens** 값

3

웹 토큰 계산기 — CLAUDE.md가 실제로 몇 토큰인지 확인

token-counter.app/anthropic · claude-tokenizer.vercel.app

툴은 필요한 것만 연동하기

서브에이전트·MCP·혹·커스텀 커맨드를 이것저것 설치하다 보면,
실수로 전역 레벨에서 연동해버리면 모든 프로젝트에서 컨텍스트를 차지합니다.

지금 당장 점검하세요

- 현재 사용 중인 확장 기능 목록 확인
- 최근 2주 동안 실제로 사용한 것만 남기기
- "나중에 쓸 수도 있으니까" 는 토큰 낭비의 핑계

서브에이전트 비용 제대로 인식하기

에이전트 워크플로우는 단일 세션 대비 최대 **10배** 더 토큰을 사용해요.

각 서브에이전트가 깨어날 때마다 **자신만의 전체 컨텍스트를 다시 로드**하기 때문입니다.

현명하게 쓰는 방법

- 단발성 작업에만 활용
- 탐색·리서치 작업 위임
- 20줄 이상의 분석 작업

모델 선택 팁

- 단순 작업은 **Haiku** 로 위임
- Haiku = Sonnet·Opus보다 훨씬 저렴
- 에이전트 팀은 **정말 필요할 때만**

지금 당장 시작하세요

부담스러우면 이 3가지부터

1

`.claudeignore` 파일 만들기

한 번 만들면 계속 씁니다

2

CLAUDE.md 다이어트

200줄 넘어가는 분들, 지금 바로 확인

3

MCP 서버 정리

`/mcp` 실행 → 안 쓰는 것 바로 끄기

CLAUDE CODE · 토큰 절약 시리즈 3편

클로드 코드 토큰 절약 **고급편**

초급 · 중급편을 다 보셨나요?

오늘은 마지막 — 고급 팁 **10**가지입니다.



이 시리즈 총 정리

19 + 23 + 10
초급 중급 고급

총 52가지

전부 다 한꺼번에 적용하려 하지 마세요.
단계적으로 익혀가면 무리 없이 할 수 있어요.

TIP 01 · 코드베이스 사전 인덱싱 (QMD)

"인증 관련 코드 어디 있어?" 물어보면 무슨 일이 생기는지 아세요?

1. glob으로 전체 파일 목록 읽기



2. grep으로 키워드 검색



3. 파일 통째로 읽기

의미 있는 단계는 3번뿐이에요. 앞의 1·2번은 탐색에 낭비되는 토큰입니다.

이걸 매번 질문할 때마다 반복한다고 생각해보세요.

qmd를 쓰면 이 탐색 과정을 **통째로 건너뛰니다**. 프로젝트의 모든 파일을 미리 색인해놓고 클로드가 검색 엔진처럼 바로 원하는 코드를 찾을 수 있게 해주는 거예요. 설치 후 CLAUDE.md에 딱 한 줄만 추가하면 돼요.

"파일을 읽기 전에 항상 qmd로 먼저 검색하라."

TIP 01 · QMD 실제 절감 효과

작업	기존 토큰	qmd 적용 후	절감률
인증 로직 탐색	2,700	250	91%
결제 버그 수정	3,950	400	90%
DB 스키마 파악	6,000	400	93%

평균 절감률

92%

탐색 토큰 기준

qmd 설치 → 프로젝트 인덱싱 → Claude Code에 MCP 서버로 연결. **5분이면 끝이에요.**

프로젝트가 커질수록 효과가 극대화됩니다.

TIP 02 · 파일 재읽기 방지 훅 (READ-ONCE)

같은 파일을 반복해서 읽는 낭비를 막아요

클로드는 세션 내에서 같은 파일을 반복해서 읽어요. `package.json` 을 읽고, 변경하고, 또 읽어서 확인하고, 연관 파일 작업할 때 또 읽어요. 매번 읽을 때마다 그 파일의 **전체 토큰 비용이 발생합니다.**

현재 상태

Read 도구 토큰의 40~60%가 중복 재읽기에 낭비

read-once 적용 후

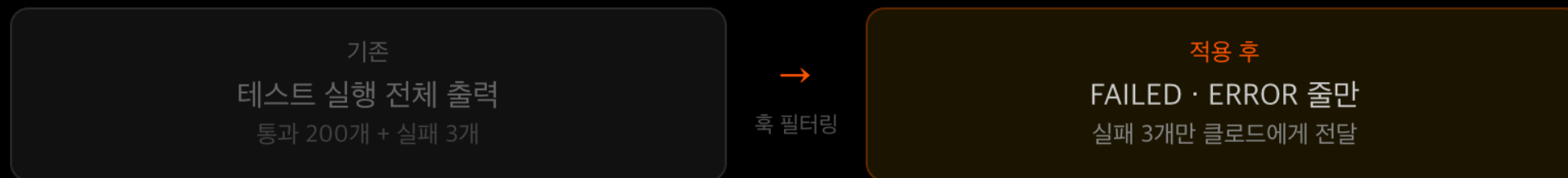
이미 읽은 파일을 추적 → 중복 재읽기 차단 → **60~90% 절감**

diff 모드도 있어요 — 파일 읽기를 완전히 차단하는 대신 마지막으로 읽은 이후 **변경된 줄만 반환**하는 방식. 편집 내용을 확인해야 할 때 유용합니다.

TIP 03 · 흑으로 로그 전처리

클로드에게 필요한 건 **에러 메시지**뿐이에요

테스트를 실행하거나 빌드를 돌리면 **전체 출력이 컨텍스트 창으로 들어와요**. 테스트 수백 개의 결과, 빌드 경고 메시지들이 전부 토큰이 되는 거예요. 통과된 테스트 200개 결과는 클로드한테 필요 없어요.



- **빌드 로그** — WARNING, ERROR 포함된 줄만 추출
- **git log** — 최근 10개로 자동 제한
- **verbose 출력** — head나 tail로 자동 캡

TIP 04 · 환경 변수로 비용 직접 제어

명령어 없이 환경 변수로 바로 제어

```
export DISABLE_NON_ESSENTIAL_MODEL_CALLS=1
```

백그라운드 모델 호출 비활성화 - 핵심 워크플로우에는 영향 없음

```
export DISABLE_COST_WARNINGS=1
```

비용 경고 메시지 끄기 - ccusage로 사용량 파악 후 적용 권장

```
export DISABLE_PROMPT_CACHING=0
```

전체 비활성화 (디버깅·벤치마킹 시에만)

```
export DISABLE_PROMPT_CACHING_SONNET=1 # 모델별 선택 비활성화
```

프롬프트 캐싱은 반복 요청 비용을 많이 절감해줘요. 특별한 경우가 아니라면 **켜두는 게 유리합니다.**

TIP 05 · MCP 툴 설명 다이어트

도구 설명이 길면 그게 그대로 토큰이에요

Before — ~300 토큰

"Reads the complete contents of a file at the specified path. Supports relative and absolute paths. Returns the file content as a string. Useful when you need to inspect, analyze, or modify..."

After — ~20 토큰

"Read file contents at path. Returns string."

AI가 이해할 수 있는 최소한만 남기기

$$\begin{array}{ccccccc} \text{도구 1개당} & & & \text{도구} & & \text{메시지마다} & \\ \text{60~80 토큰} & \times & \text{50개} & = & \text{3,000~4,000} & & \\ \text{절약} & & & & \text{토큰 절감} & & \end{array}$$

실제로 쓰는 도구만 노출하세요

특정 MCP 서버에서 실제로 쓰는 도구가 전체의 일부밖에 안 되는 경우가 많아요. 서버를 연결하면 **전체 도구 정의가 다 로드**되니까요.



- 쓰고 있는 MCP 서버 문서나 소스코드에서 필터링 옵션 확인
- 오픈소스라면 직접 포크해서 안 쓰는 도구 제거 또는 필터링 레이어 추가
- 모든 서버가 지원하지 않지만 **대형 서버들은 확인해볼 가치 있음**

TIP 07 · 에이전트별 최소 도구 세트 분리

단일 세션에 모든 도구를 몰아넣지 마세요

코드 리뷰

웹 검색 도구
필요 없어요

고객 데이터

파일시스템 도구
필요 없어요

배포 담당

DB 조회 도구
필요 없어요

역할별 도구 최소화로 두 가지 이점이 동시에 생겨요.

토큰 절감 — 불필요한 도구 정의가 컨텍스트에 들어오지 않음

오작동 리스크 감소 — 코드 리뷰 에이전트가 DB에 직접 접근할 수 없으니 실수 방지

하나의 터미널에서 모든 작업? 컨텍스트가 오염돼요

피쳐 개발 하다가 리팩토링 하고, 다시 버그 픽스하면 서로 다른 작업의 잔재들이 컨텍스트에 뒤섞입니다.

터미널 1

기능 개발
기능 구현 관련 컨텍스트만

터미널 2

리팩토링
구조 개선 관련 맥락만

터미널 3

디버깅
오류 해결·로그 분석만

- **컨텍스트 충돌 없음** — 각 터미널이 자신의 작업에만 집중
- **메모리 효율 향상** — 필요한 정보만 유지해 컨텍스트가 가볍게 유지
- **병렬 작업 가능** — 한 터미널 개발 중, 다른 터미널에서 디버깅 동시 진행

TIP 10 · 🗣️ 캐슬 AI

유튜브 구독하고 클로드 코드 꿀팁 많이 얻어가기

감사합니다 사랑합니다 🙏❤️



좋아요



댓글



구독



알림설정

 단계적 적용 순서

이 순서로 가면 무리 없이 적용할 수 있어요

1 초급 습관화

/clear 습관 · 소넷 모델 기본 사용 — 이것만 해도 체감이 달라요

2 중급 환경 정비

.claudeignore 만들기 · CLAUDE.md 다이어트 · MCP 서버 정리

3 고급 도구 적용

qmd 설치 · read-once 후 적용 · 멀티 터미널 운용



MAX 플랜 전에 52가지 먼저

이것만 해도 지금 플랜으로 충분히 많은 걸 할 수 있어요

Claude Code

MCP→CLI 전환

핵심 가이드

토큰 17배 절감, 성공률 100%로 가는 실전 전환법

heyjames.ai

2026년 4월 · 헤이제임스

목차

Part 1 — MCP의 구조적 문제 이해하기

- 1.1 MCP란 무엇인가
- 1.2 토큰 블랙홀: 스키마 로딩의 비밀
- 1.3 안정성 문제: 타임아웃과 좀비 프로세스
- 1.4 인증 지옥: 이중 인증의 함정
- 1.5 비용 비교: 벤치마크 데이터

Part 2 — CLI 대체 도구 실전 가이드

- 2.1 MCP → CLI 대체 목록 총정리
- 2.2 GitHub: gh CLI 완전 정복
- 2.3 HTTP API: curl + jq 조합
- 2.4 클라우드: AWS, Docker, Kubernetes CLI
- 2.5 CLAUDE.md 작성법

Part 3 — 고급 최적화 & 하이브리드 전략

- 3.1 MCP가 여전히 필요한 경우
- 3.2 ENABLE_TOOL_SEARCH 숨겨진 설정
- 3.3 MCP Gateway로 스키마 필터링
- 3.4 전환 체크리스트

MCP의 구조적 문제 이해하기

MCP(Model Context Protocol)가 왜 토큰을 낭비하고 불안정한지, 구조적 원인을 이해합니다.

1.1 MCP란 무엇인가

MCP(Model Context Protocol)는 AI 모델에게 외부 도구를 연결하는 표준 규격입니다. GitHub에서 코드를 가져오거나, 데이터베이스를 조회하거나, API를 호출할 때 MCP 서버를 통해 처리합니다.

CLI(Command Line Interface)는 터미널에서 직접 실행하는 명령어 도구입니다. `gh`, `curl`, `docker`, `kubectl` 등 개발자가 이미 매일 사용하는 도구들이 여기에 해당합니다.

핵심 차이: MCP는 모든 도구 스키마를 미리 컨텍스트에 로드하지만, CLI는 필요할 때 명령어 하나만 실행합니다.

1.2 토큰 블랙홀: 스키마 로딩의 비밀

MCP의 가장 큰 문제는 **스키마 선로딩**입니다. MCP 서버를 연결하면 Claude Code가 시작되는 순간 해당 서버의 **모든 도구 정의**를 컨텍스트 윈도우에 올립니다.

MCP 서버	도구 수	소비 토큰	컨텍스트 비율
GitHub MCP	43개	~55,000	~28%
Atlassian MCP	73개	~45,000	~23%
Microsoft Graph MCP	28개+	~28,000	~14%
Chrome 관련 MCP 2개	20개+	~31,700	~16%

주의: MCP 서버 2~3개만 연결해도 컨텍스트의 40~50%가 스키마에 소비됩니다. 사용자가 한 마디도 입력하기 전예요.

벤치마크: 동일 작업의 토큰 비교

"이 레포지토리의 주요 프로그래밍 언어는?"이라는 간단한 질문에 대한 결과:

CLI (gh CLI)

1,365 토큰

명령어 하나 실행 → 결과 반환

MCP (GitHub MCP)

44,026 토큰

43개 도구 스키마 + 실행 = 32배 차이

1.3 안정성 문제: 타임아웃과 좀비 프로세스

GitHub Issue #15945에 보고된 실제 사례: MCP 서버 사용 중 Claude Code가 **16시간 동안 완전히 멈춤**. 에러 메시지 없음, 경고 없음, 로그 없음. 확인 결과 좀비 프로세스가 70개 이상 쌓여 있었습니다.

성공률 벤치마크 (동일 작업 25회 반복)

방식	성공	실패	성공률
CLI	25회	0회	100%
MCP	18회	7회	72%

추가 문제: MCP의 타임아웃 설정 기능이 존재하지만, SSE 연결에서는 설정값이 무시되는 버그가 보고되어 있습니다 (Issue #3033, #20335).

1.4 인증 지옥: 이중 인증의 함정

GitHub Issue #19281: 한 개발자가 GitHub MCP 인증 설정에 2시간을 소비한 후 결국 실패.

- 1 gh auth login 으로 이미 로그인 완료 상태
- 2 MCP는 기존 gh auth 인증을 무시하고 별도 PAT 토큰 요구
- 3 OAuth 토큰(gh*_)과 PAT 토큰(ghp*_)이 호환 안 됨 — 미문서화
- 4 에러 메시지: "Authentication Failed: Bad credentials" — 원인 불명

1.5 비용 비교: 벤치마크 데이터

월 10,000건 작업 기준 (Claude Sonnet 4 가격, \$3/M input, \$15/M output):

방식	월 비용	배수
CLI	~\$3.20	1x (기준)
MCP via Gateway	~\$5.00	1.6x
MCP 직접 연결	~\$55.20	17.3x

PART 2

CLI 대체 도구 실전 가이드

주요 MCP 서버를 어떤 CLI로 교체하는지, 구체적인 명령어와 함께 알아봅니다.

2.1 MCP → CLI 대체 목록 총정리

MCP 서버	CLI 대체	설치
GitHub MCP (43 tools)	<code>gh</code>	<code>brew install gh</code>
HTTP/Fetch MCP	<code>curl</code> + <code>jq</code>	기본 내장 / <code>brew install jq</code>
AWS MCP	<code>aws</code>	<code>brew install awscli</code>
Kubernetes MCP	<code>kubectl</code>	<code>brew install kubectl</code>
Docker MCP	<code>docker</code>	Docker Desktop 포함
Google Cloud MCP	<code>gcloud</code>	<code>brew install google-cloud-sdk</code>
Terraform MCP	<code>terraform</code>	<code>brew install terraform</code>
Jira MCP	<code>jira</code>	<code>brew install jira-cli</code>
파일 시스템 MCP	<code>find</code> , <code>grep</code> , <code>cat</code>	기본 내장

2.2 GitHub: gh CLI 완전 정복

`gh` CLI는 GitHub의 공식 CLI 도구로, MCP가 제공하는 43개 도구의 기능을 모두 대체할 수 있습니다.

자주 쓰는 명령어

```
# PR 관련
gh pr list
```

```
# PR 목록
```

```

gh pr create --title "제목" --body "내용" # PR 생성
gh pr view 123 # PR 상세 보기
gh pr merge 123 # PR 병합

# Issue 관련
gh issue list --label "bug" # 이슈 목록
gh issue create --title "제목" # 이슈 생성

# API 직접 호출 (MCP로는 불가능한 커스텀 요청)
gh api repos/{owner}/{repo}/actions/runs # 워크플로우 조회
gh api graphql -f query='{ viewer { login } }' # GraphQL

```

장점: `gh auth login` 한 번이면 인증 완료. MCP처럼 별도의 PAT 토큰 설정이 필요 없습니다.

2.3 HTTP API: curl + jq 조합

외부 API를 호출하는 Fetch/HTTP MCP는 `curl` 과 `jq` 로 완전히 대체 가능합니다.

```

# JSON API 호출 + 원하는 필드만 추출
curl -s https://api.example.com/data | jq '.results[] | {name, status}'

# POST 요청
curl -s -X POST https://api.example.com/items \
  -H "Content-Type: application/json" \
  -d '{"name": "test"}' | jq '.id'

# 인증이 필요한 API
curl -s -H "Authorization: Bearer $TOKEN" \
  https://api.example.com/protected | jq '.'

```

Unix 파이프의 힘: `curl` 과 `jq` 를 파이프(`|`)로 연결하면, MCP에서는 커스텀 서버 코드를 짜야 하는 복잡한 작업도 한 줄로 처리할 수 있습니다.

2.4 클라우드: AWS, Docker, Kubernetes CLI

AWS CLI

```

# EC2 인스턴스 목록
aws ec2 describe-instances --output json | jq '.Reservations[].Instances[] | {id: .InstanceId, state: .State.Name}'

```



```
# S3 버킷 목록  
aws s3 ls
```

Docker CLI

```
# 컨테이너 상태 확인  
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

```
# 로그 확인  
docker logs --tail 50 my-container
```

kubectl

```
# Pod 상태 확인  
kubectl get pods -o json | jq '.items[] | {name: .metadata.name, status: .status.phase}'
```

```
# 리소스 적용  
kubectl apply -f deployment.yaml
```

2.5 CLAUDE.md 작성법

프로젝트 루트에 `CLAUDE.md` 파일을 만들어 사용 가능한 CLI 도구를 명시하면, Claude가 자동으로 해당 도구를 활용합니다.

```
# Available CLI Tools
```

```
## GitHub  
- `gh` CLI로 GitHub 작업 수행 (PR, Issue, API 호출)  
- 인증: `gh auth login`으로 설정 완료
```

```
## API 호출  
- `curl`로 외부 API 호출, `jq`로 JSON 파싱  
- 인증 토큰은 환경변수로 관리
```

```
## 인프라  
- `docker`로 컨테이너 관리  
- `kubectl`로 Kubernetes 클러스터 관리  
- `aws` CLI로 AWS 리소스 관리
```

```
## 참고  
- CLI 사용법이 궁금하면 `--help` 플래그 사용  
- 복잡한 JSON 파싱은 `jq`로 처리
```

핵심: CLAUDE.md에 도구 목록만 적으면 됩니다. Claude는 학습 데이터에서 이미 이 도구들의 사용법을 충분히 알고 있습니다.

PART 3

고급 최적화 & 하이브리드 전략

MCP를 완전히 버릴 수 없는 경우를 위한 최적화 방법과 전환 체크리스트를 제공합니다.

3.1 MCP가 여전히 필요한 경우

CLI로 100% 전환이 불가능한 경우가 있습니다:

상황	이유	예시
CLI 없는 서비스	공식 CLI 도구가 존재하지 않음	Figma, Notion, Slack API
멀티 테넌트 OAuth	사용자별 권한 분리 필요	SaaS 플랫폼에서 고객별 접근
비개발자 환경	터미널 접근 불가	Claude Desktop 사용자
보안 규정	엄격한 입력 검증 필요	금융/의료 규제 환경

3.2 ENABLE_TOOL_SEARCH 숨겨진 설정

Claude Code에는 MCP 도구를 지연 로딩하는 숨겨진 설정이 있습니다. 이를 활성화하면 **30,000+ 토큰을 절약**할 수 있습니다.

설정 방법

```
# ~/.claude/settings.json 파일을 열고 추가:
{
  "env": {
    "ENABLE_TOOL_SEARCH": "auto:0"
  }
}
```

동작 원리: 이 설정을 적용하면 MCP 도구를 시작 시 모두 로드하지 않고, Claude가 필요할 때 검색해서 사용합니다. 기존: 모든 스키마 선로딩 → 변경: 필요한 도구만 온디맨드 로딩.

3.3 MCP Gateway로 스키마 필터링

MCP를 써야 하는 경우에도, MCP Gateway를 중간에 두면 토큰 오버헤드를 최대 90% 줄일 수 있습니다.



Gateway가 하는 일:

- 1 스키마 필터링: 43개 도구 중 실제 사용하는 5~10개만 노출
- 2 설명 압축: 긴 파라미터 설명을 필수 정보만 남기고 축약
- 3 캐싱: 자주 사용하는 결과를 캐싱하여 반복 호출 방지

토큰 절감 효과 (Intune 환경 벤치마크)

방식	50개 디바이스 검사	배수
MCP 직접	145,000 토큰	35x
MCP + Gateway	~15,000 토큰	3.6x
CLI 래퍼	4,150 토큰	1x

3.4 전환 체크리스트

아래 체크리스트를 따라 단계적으로 전환하세요:

- 1 Claude Code에서 `/mcp` 입력 → 현재 연결된 MCP 서버 목록 확인
- 2 각 MCP 서버에 대해 CLI 대체 가능 여부 확인 (Part 2의 대체 목록 참고)
- 3 CLI로 대체 가능한 MCP 서버 비활성화
- 4 프로젝트 루트에 `CLAUDE.md` 작성 — 사용 가능한 CLI 도구 목록 명시
- 5 CLI가 없는 MCP만 남긴 경우 → `ENABLE_TOOL_SEARCH` 설정 적용
- 6 남은 MCP에 Gateway 도입 검토 → 스키마 필터링으로 토큰 추가 절감

예상 효과: 위 단계를 모두 적용하면 토큰 소비량 80~95% 감소, 안정성 100%, 월 비용 10배 이상 절감이 가능합니다.

everything-claude-code

핵심 가이드

14만 스타 플러그인으로 Claude Code 비용 80% 줄이기

2026년 4월 | v1.10.0 기준

Context Rot • Token Diet • Continuous Learning • AgentShield • Cross-Harness

목차

Part 1. 시작하기 — 왜 이 도구인가

- › everything-claude-code란?
- › 만든 사람과 검증된 권위
- › Context Rot — 클로드가 멍청해지는 진짜 이유
- › settings.json 세 줄로 비용 80% 절감
- › 모델 선택 의사결정 표

Part 2. 실전 활용 — 워크플로와 설치

- › /clear vs /compact — 컨텍스트 다이어트
- › MCP 함정 — 200K가 70K가 되는 이유
- › 3단계 설치 가이드
- › 처음 깔아야 할 코어 스킬 5개
- › 핵심 슬래시 명령어 워크플로

Part 3. 고급 기능 패턴

- › Continuous Learning v2 — 메모리 학습 시스템
- › instinct → skill 자동 승격
- › AgentShield — AI 에이전트 보안 스캐너
- › 크로스 하네스 — 6개 톨 한 번에
- › 팀 단위 적용 패턴

부록

- › 공식 링크 모음
- › 액션 아이템 3가지
- › 함정과 주의사항

PART 1

시작하기 — 왜 이 도구인가

Context Rot의 정체부터 settings.json 세 줄로 비용을 80% 줄이는 법까지

everything-claude-code란?

everything-claude-code는 Claude Code, Cursor, Codex, OpenCode, Antigravity, Gemini CLI 등 주요 AI 코딩 하네스에서 동작하는 에이전트 하네스 성능 최적화 시스템입니다. 단순 설정 모음이 아니라 에이전트, 스킬, 룰, MCP 설정, 메모리 학습 시스템을 통째로 묶은 플러그인입니다.

지표	값
GitHub Stars	143,075개 (2026-04-07 기준)
Forks	21,793개
저장소 생성일	2026-01-18 (약 80일 만에 14만 스타 돌파)
구성 요소	38 agents / 156 skills / 72 commands / 34 rules / 14 MCP servers
테스트	1,282 tests / 98% coverage
지원 하네스	Claude Code, Cursor, Codex (앱+CLI), OpenCode, Antigravity, Gemini CLI
라이선스	MIT

만든 사람과 검증된 권위

저장소를 만든 **Affaana Mustafa**는 샌프란시스코 기반 AI/금융 스타트업 Itô의 공동창업자입니다. 2025년 9월 Anthropic x Forum Ventures가 뉴욕에서 연 해커톤에서 100팀 중 1등을 했고, 동료와 둘이서 zenith.chat이라는 서비스를 Claude Code로 8시간 만에 만들어 상금 15,000달러어치 Anthropic API 크레딧을 받았습니다. 그가 X에 올린 Claude Code 가이드 트윗은 누적 1천만 뷰를 넘었습니다.

핵심: 어디서 한 번 만들어 본 게 아닙니다. 매일 Claude Code로 먹고사는 사람이 10개월 동안 매일 다듬은 워크플로를 통째로 오픈소스로 공개한 것입니다.

Context Rot — 클로드가 멍청해지는 진짜 이유

Claude Code를 한참 쓰다 보면 갑자기 출력 품질이 무너지는 순간이 옵니다. 같은 실수를 반복하고, 알려준 규칙을 까먹고, 시키지도 않은 파일을 건드립니다. 이건 버그가 아니라 **Context Rot**이라는 이름까지 붙은 구조적 현상입니다. 무서운 건 크래시가 나는 게 아니라 그냥 왠지 멍청해진다는 점이에요. 사용자가 잘못된 줄 알게 되어 잡기가 어렵습니다.

네 가지 원인

- 1 **어텐션 희석(Attention Dilution)** — 문맥이 길어질수록 클로드가 중간에 있는 정보를 잘 못 떠올립니다.
- 2 **명령 충돌** — 누적된 지시문이 서로 모순돼서 어느 쪽을 따라야 할지 헷갈립니다.
- 3 **토큰 예산 압박** — 부풀어 오른 시작 파일이 정작 작업할 토큰을 다 잡아먹습니다.
- 4 **관련성 미스매치** — 모든 파일이 매번 다 로드되니 작업과 무관한 정보까지 머릿속에 들어가 있습니다.

실전 임계: 한 파일이 **2,000 ~ 3,000 토큰**(약 1,500단어)을 넘으면 점검 신호로 보세요. CLAUDE.md가 너무 두꺼우면 그 자체로 멍청해지는 원인이 됩니다.

settings.json 세 줄로 비용 80% 절감

홈 디렉토리의 `~/.claude/settings.json` 파일을 열고 딱 세 가지를 추가하면 됩니다. Affaan이 직접 측정한 권장 세팅입니다.

```
{
  "model": "sonnet",
  "env": {
    "MAX_THINKING_TOKENS": "10000",
    "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE": "50",
    "CLAUDE_CODE_SUBAGENT_MODEL": "haiku"
  }
}
```

설정	기본값	권장값	효과
----	-----	-----	----

model	opus	sonnet	약 60% 비용 절감 (코딩 작업 80%는 sonnet으로 충분)
MAX_THINKING_TOKENS	31,999	10,000	숨은 thinking 비용 약 70% 절감
CLAUDE_AUTOCOMPACT_PCT_OVERRIDE	95	50	긴 세션 품질 개선 — 더 일찍 압축
CLAUDE_CODE_SUBAGENT_MODEL	(상위 모델)	haiku	서브 에이전트 비용 약 80% 절감

누적 효과: 세 가지를 합치면 기본 설정 대비 **80% 이상 비용 절감**이 가능합니다. 같은 작업이 한 번은 4달러, 다른 한 번은 60센트로 끝나는 식입니다.

모델 선택 의사결정 표

상황	모델	이유
일상 코딩, 리뷰, 테스트 작성	Sonnet (기본)	품질/비용 균형 최적
아키텍처 설계, 깊은 디버깅	Opus (/model opus)	복잡한 추론에 한해 일시 전환
단순 조회, 반복 작업	Haiku (서브에이전트)	가장 저렴, 워커 역할

PART 2

실전 활용 — 워크플로와 설치

설정만으로는 부족하다. 손가락 습관과 설치 순서까지 한 번에 정리

/clear vs /compact — 컨텍스트 다이어트

Claude Code에는 두 가지 컨텍스트 정리 명령이 있는데, 대부분 차이를 모르고 씁니다. 잘 쓰면 같은 세션에서 토큰 효율이 두 배 이상 나옵니다.

/clear (무료, 즉시)

언제: 무관한 작업 사이

효과: 컨텍스트 통째로 비움

예시: 버그 수정 끝 → 새 기능 시작 직전

/compact (요약 압축)

언제: 마일스톤 사이

효과: 컨텍스트 요약 후 압축 보존

예시: 리서치 끝 → 구현 시작 직전

절대 금지: 구현 도중에 `/compact` 하지 마세요. 변수명, 파일 경로, 부분 상태를 다 잃어버립니다. 자연스러운 끊김 지점에 서만 사용하세요.

컴팩트 적정 시점 5가지

- 1 리서치/탐색이 끝나고 구현 들어가기 직전
- 2 한 마일스톤이 완료되고 다음 마일스톤 시작 직전
- 3 디버깅이 끝나고 기능 작업으로 돌아갈 때
- 4 실패한 접근을 포기하고 새 접근을 시도할 때
- 5 `/cost` 로 확인했을 때 컨텍스트 사용량이 50%를 넘었을 때

MCP 함정 — 200K가 70K가 되는 이유

MCP 서버를 너무 많이 켜두면 안 됩니다. MCP 하나하나가 도구 설명을 토큰으로 잡아먹기 때문에, 다 켜두면 **200,000 토큰** 짜리 컨텍스트 윈도우가 **70,000 토큰**까지 줄어듭니다.

권장 임계: 프로젝트당 MCP는 **10개 이하**, 활성 도구는 **80개 이하**로 유지하세요.

프로젝트별 MCP 비활성화

```
// 프로젝트의 .claude/settings.json
{
  "disabledMcpServers": ["supabase", "railway", "vercel"]
}
```

3단계 설치 가이드

1단계: settings.json 세 줄



2단계: 플러그인 설치



3단계: rules 수동 설치

1단계 — 오늘 당장 적용 (1분)

위 Part 1의 settings.json 세 줄을 추가합니다. 이것만으로도 다음 달 청구서가 줄어듭니다.

2단계 — 플러그인 설치 (1분)

```
# Claude Code 안에서 실행
/plugin marketplace add https://github.com/affaan-m/everything-claude-code
/plugin install ecc@ecc
```

3단계 — rules 수동 설치 (5분)

플러그인 시스템은 rules를 자동 배포할 수 없으므로 별도로 깔아야 합니다.

```
# 클론 후 설치
git clone https://github.com/affaan-m/everything-claude-code.git
cd everything-claude-code
npm install

# macOS / Linux
./install.sh --profile full

# 또는 언어별로
./install.sh typescript python golang swift php
```

```
# Windows PowerShell
.\install.ps1 --profile full
```

주의: Claude Code **v2.1.0** 이상이 필요합니다. `claude --version` 으로 확인하세요. 그리고 `.claude-plugin/plugin.json` 에 `"hooks"` 필드를 직접 추가하지 마세요. 자동 로드와 충돌해서 "Duplicate hooks file" 에러가 납니다.

처음 깔아야 할 코어 스킬 5개

156개 스킬을 다 설치하지 마세요. 다음 다섯 개부터 시작하시고, 필요할 때마다 하나씩 추가하면 됩니다.

스킬	역할
<code>search-first</code>	코딩 전에 먼저 리서치하는 워크플로 (가장 중요)
<code>tdd-workflow</code>	테스트 주도 개발 — 실패 테스트 먼저 작성
<code>strategic-compact</code>	적절한 시점에 자동으로 <code>/compact</code> 제안
<code>verification-loop</code>	build, test, lint, typecheck, security를 한 번에 검증
<code>security-review</code>	OWASP Top 10 기반 보안 체크리스트

핵심 슬래시 명령어 워크플로

새 기능 개발



`/plan` 이 planner 에이전트로 청사진을 그리고, `/tdd` 가 tdd-guide 에이전트로 테스트 먼저 강제하며, `/code-review` 가 code-reviewer로 회귀를 잡습니다.

버그 수정



프로덕션 출시 전



PART 3

고급 기능 패턴

14만 명이 이걸 즐겨찾기한 진짜 이유 — 메모리 학습과 보안

Continuous Learning v2 — 메모리 학습 시스템

everything-claude-code의 진짜 핵심은 토큰 절감이 아니라 **Continuous Learning v2**(지속 학습 2.0) 시스템입니다. Claude Code가 도구를 호출하기 직전(`PreToolUse`)과 직후(`PostToolUse`)에 훅이 자동으로 발동돼 사용자 작업을 100% 관찰하고 반복 패턴을 학습합니다.

학습 단위: instinct

학습된 한 패턴을 **instinct**(본능)라고 부르며, 각 instinct에는 **0.3 ~ 0.9** 사이의 신뢰도 점수가 붙습니다. 자주 발견되고 일관성이 높을수록 점수가 올라갑니다.

예시: "이 사용자는 새 React 컴포넌트를 만들 때 항상 `components/ui/` 폴더 아래에 같은 패턴으로 만든다" → 신뢰도 0.85짜리 instinct로 저장.

instinct → skill 자동 승격

관련된 instinct가 **3개 이상** 모이면 `/evolve` 명령으로 자동으로 재사용 가능한 skill 모듈로 승격됩니다. 이게 바로 클로드가 가르친 걸 영구적으로 기억하게 만드는 매커니즘입니다.

PreToolUse 훅 관찰



instinct 저장 (신뢰도 0.3-0.9)



3개 이상 → skill 승격

핵심 명령어

명령	역할
<code>/instinct-status</code>	학습된 instinct 목록과 신뢰도 확인

<code>/instinct-export</code>	내 학습 결과를 파일로 내보내기 (팀 공유용)
<code>/instinct-import <file></code>	동료의 학습 결과를 가져오기
<code>/evolve</code>	관련 instinct 묶음을 skill로 자동 승격
<code>/prune</code>	30일 TTL이 지난 pending instinct 정리

한국 회사 적용: 한 번 가르친 우리 회사 코딩 컨벤션을 Claude가 영구히 기억하고, `/instinct-export` & `/instinct-import` 로 신입한테 그대로 넘겨줄 수 있습니다. 사람이 아닌 도구에 노하우를 축적하는 첫 번째 실용적 사례입니다.

AgentShield — AI 에이전트 보안 스캐너

같은 사람(Affaan)이 만든 보안 스캐너로, 설치 없이 한 줄로 실행 가능합니다. Claude Code 설정 전체를 5가지 카테고리로 스캔합니다.

```
# 설치 없이 즉시 스캔
npx ecc-agentshield scan

# 안전한 이슈 자동 수정
npx ecc-agentshield scan --fix

# Opus 4.6 3에이전트 적대적 검증
npx ecc-agentshield scan --opus --stream

# Claude Code 안에서 실행
/security-scan
```

스캔 5개 카테고리

카테고리	검사 내용
시크릿 탐지	14가지 패턴으로 노출된 API 키, 토큰 검출
권한 감사	너무 열려 있는 도구 권한 식별
혹 인젝션 분석	혹 명령어의 prompt injection 취약점
MCP 서버 리스크	위험한 MCP 서버 프로파일링

--opus 플러그: 적대적 검증 파이프라인



Claude Opus 4.6 에이전트 3마리가 각각 익스플로잇 탐색, 방어 평가, 우선순위 리스크 종합을 수행합니다. 단순 패턴 매칭이 아닌 적대적 추론입니다. 출력은 터미널(A-F 등급), JSON, Markdown, HTML로 받을 수 있고, critical 발견 시 exit 2로 CI 빌드 게이트에 활용 가능합니다.

스펙: 1,282개 테스트, 102개 정적 분석 룰, 98% 커버리지. Cerebral Valley x Anthropic 해커톤에서 빌드된 도구입니다.

크로스 하네스 — 6개 툴 한 번에

everything-claude-code는 Claude Code 전용이 아닙니다. 같은 설정으로 여섯 개의 AI 코딩 툴이 동시에 작동합니다.

툴	지원 범위	특이점
Claude Code	네이티브 (47 agents, 79 commands, 181 skills)	주 타깃
Cursor IDE	완전 지원, 15 hook events	혹 어댑터로 Claude 스크립트 재사용
Codex (앱 + CLI)	AGENTS.md + .codex 설정	3 멀티에이전트 룰 (explorer, reviewer, docs)
OpenCode	12 agents, 31 commands, 37 skills, 11 hook events	오히려 혹 이벤트가 더 많음
Antigravity	flat rules + .agent 통합	워크플로/스킬 통합
Gemini CLI	실험적 .gemini/GEMINI.md	install.sh --target gemini

핵심 아키텍처: 루트의 AGENTS.md 파일 하나가 6개 툴의 공통 진입점 역할을 합니다. DRY 어댑터 패턴으로 Cursor 혹은 Claude Code 혹은 스크립트를 그대로 재사용합니다.

팀 단위 적용 패턴

- 1 리드 개발자가 먼저 설정 — settings.json 세 줄, 코어 스킬 5개, instinct 시드 학습
- 2 **instinct export** — 1~2주 사용 후 `/instinct-export` 로 팀 노하우 파일 생성
- 3 전사 공유 — 신입 입사 시 `/instinct-import` 로 컨벤션 즉시 주입
- 4 CI 게이트 — `npx ecc-agentshield scan` 을 PR 빌드에 추가 (exit 2)
- 5 주기적 **evolve** — 한 달에 한 번 `/evolve` 로 instinct를 정식 skill로 승격

부록

레퍼런스와 액션 아이템

바로 시작할 수 있는 링크와 다음 주에 할 일 세 가지

공식 링크 모음

리소스	링크
everything-claude-code 저장소	github.com/affaan-m/everything-claude-code
Token Optimization 공식 문서	github.com/affaan-m/everything-claude-code/blob/main/docs/token-optimization.md
AgentShield 보안 스캐너	github.com/affaan-m/agentshield
저자 사이트 (Affaan Mustafa)	affaanmustafa.com
Shorthand Guide (X 트윗)	x.com/affaanmustafa/status/2012378465664745795
Longform Guide (X 트윗)	x.com/affaanmustafa/status/2014040193557471352
Security Guide (X 트윗)	x.com/affaanmustafa/status/2033263813387223421

액션 아이템 3가지

- 1 **오늘:** `~/.claude/settings.json` 에 세 줄 추가하고 `/cost` 로 비교 측정. 같은 작업을 변경 전후로 한 번씩 돌려보세요.
- 2 **이번 주:** `/plugin marketplace add` 로 플러그인 설치, 코어 스킬 5개부터 활성화. `/clear` 와 `/compact` 를 의식적으로 사용.

- 3 이번 달: Continuous Learning v2 활성화 → 1~2주 후 `/instinct-status` 로 학습 결과 확인 → `/evolve` 로 스킬 승격 → 팀에 export.

합정과 주의사항

피해야 할 다섯 가지

1. 156개 스킬 한 번에 다 깔기 — 코어 5개부터 시작하세요.
2. `multi-*` 명령 바로 사용 — 별도로 `npx ccg-workflow` 런타임 설치 필요.
3. `plugin.json`에 "hooks" 필드 추가 — 자동 로드와 충돌, "Duplicate hooks file" 에러.
4. 구현 도중 `/compact` — 변수명/파일 경로/부분 상태 손실.
5. MCP 전부 활성화 — 200K 컨텍스트가 70K로 줄어듭니다.

저자의 면책: Affaan 본인이 README에 적어둔 내용입니다. "이 설정은 내 워크플로에 맞춘 것이다. 그대로 받아쓰기보다 자신의 스택에 맞게 골라 쓰고, 안 쓰는 건 빼고, 자신의 패턴을 추가하라."

본 가이드는 everything-claude-code v1.10.0 (2026년 4월 기준)을 분석하여 작성되었습니다.
최신 정보는 공식 GitHub 저장소를 참고하세요.

heyjames.ai · AI 코딩 도구 가이드

현실

하루가 다르게 새 용어가 쏟아집니다

하네스 엔지니어링

멀티 에이전트

RAG

MCP

AGI

오케스트레이션

임베딩

툴콜링

리트리버

ASI

토큰화

...

AI 도구와 개념을 빠르게 익히는 게 어느 때보다 중요한데,
매번 언제 다 공부하나 싶어 피로하실 겁니다.

AI 학습법

새 AI 개념이 나올 때마다 흔들리지 않는 법

한 번 배우면 평생 써먹는 학습법

진단

공부는 하는데 왜 쌓이지 않을까?

- 강의·책 구매, 유튜브 검색, AI에게 질문 — 방법은 다양 하지만
- 본인만의 기준이 없어서 매번 밑 빠진 독에 물 붓기
- 조각은 많은데 어디에 끼워야 할지 모르는 상태
- 전체 그림을 모르면 퍼즐 조각을 맞출 수 없듯이

핵심 원인: 개념을 받아들이는 **순서**가 없어서예요.

프레임워크

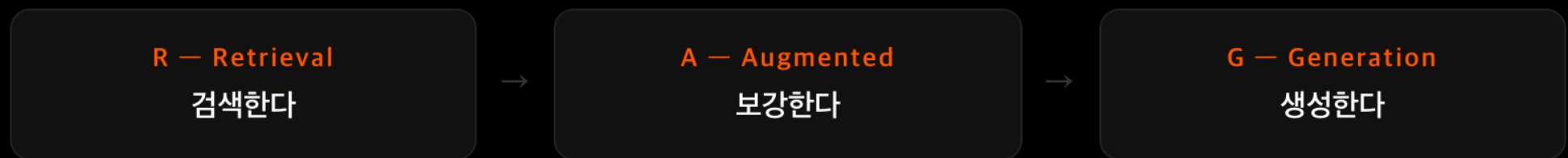
딱 3가지만 물으면 됩니다

- 1 왜 이런 이름이 붙게 되었을까?
- 2 기존에는 어떤 문제가 있었을까?
- 3 그 문제를 어떻게 해결했을까?

이 세 가지에 답할 수 있으면 — **그 기술을 정확하게 이해한 겁니다.**

예시 — RAG

RAG로 직접 보여드릴게요



이름 안에 이미 **동작 순서**가 전부 들어있습니다.

이름부터 뜯어보면 본질이 보이기 시작합니다.

질문 1

왜 이런 이름이 붙었을까?

- 단어 세 개: 검색하고, 보강하고, 생성한다
- 이름 안에 흐름이 전부 담겨 있음
- 만든 사람이 본질을 정확하게 담으려 수많은 고민 끝에 지은 이름

AI에게 질문하기: "RAG라는 기술의 어원을 공부하고 있는데,
약자를 알려주고 각 약자가 의미하는 것을 설명해줘"

주도적으로 질문하면 이해가 훨씬 풍부하고 까먹지 않습니다.

질문 2

기존에는 어떤 문제가 있었을까?

도구는 항상 문제를 해결하기 위해 존재합니다. 지우개는 잘못 쓴 걸 지우려고, 냉장고는 음식이 상하는 걸 막으려고 나왔죠.

기존의 문제

ChatGPT는 인터넷에서 배운 것만 알아요.
우리 회사 규정, 사내 문서, 어제 올라온 공지사항은 모릅니다.
학습 데이터에는 시간 한계도 있어요.

한 줄 정리

AI가 내 상황을 모른다.
이게 문제였어요.

그래서 검색하고, 보강하고, 생성하는 방법이 필요했던 겁니다.

질문 3

그 문제를 어떻게 해결했을까?

- 1 질문이 들어오면 AI가 바로 답하지 않는다
- 2 관련 문서를 먼저 검색한다 (사내 규정, 공지사항, 최신 자료)
- 3 그 문서를 질문과 함께 AI에게 넘겨준다
- 4 AI는 그걸 보고 정확하게 답변을 생성한다

그래서 이름이 **검색하고(R)**, **보강하고(A)**, **생성한다(G)**인 겁니다.

〃

R, A, G 그냥 외우면 반드시 까먹어요.

이름을 뜯어보고, 문제를 알고,

해결 방식까지 따라오면

까먹으려야 까먹을 수가 없어요.

3가지 질문으로 본질이 잡혔습니다

- 1 **이름:** 검색하고, 보강하고, 생성한다 — 동작 순서가 이름에 담김
- 2 **문제:** AI가 학습하지 않은 정보(사내 문서, 최신 자료)는 답할 수 없음
- 3 **해결:** 질문마다 필요한 문서를 먼저 찾아서 함께 넘겨주는 방식

이 틀은 어떤 개념이 나와도 그대로예요.

하네스 엔지니어링이든, MCP든, 오케스트레이션이든, 앞으로 나올 용어든 전부요.

마인드셋

새 개념이 나오면 설레야 합니다

새 용어가 나왔을 때 무작정 강의나 책을 구매하지 마세요.
이 세 가지부터 먼저 던져보세요.

왜 이런 이름이 붙었을까?

기존에 어떤 문제가 있었을까?

그 문제를 어떻게 해결했을까?

아무리 어려운 개념이어도 뼈대가 잡힙니다.

뼈대가 잡히면 살은 자연스럽게 붙어요.

속제

아래 용어들로 직접 해보세요

모르는 게 있다면 3가지 질문 순서대로 공부해보세요.

RAG ✓

하네스 엔지니어링

에이전트

멀티에이전트

오케스트레이션

MCP

AGI

ASI

LLM

클로즈드 모델

토큰화

임베딩

리트리버

리랭커

툴콜링

슬라이드 PDF 다운로드 → 댓글의 오픈 카톡방 참고

마무리

한 번 배우면 평생 써먹는 3가지 질문

- 1 왜 이런 이름이 붙었을까?
- 2 기존에 어떤 문제가 있었을까?
- 3 그 문제를 어떻게 해결했을까?

도움이 되셨다면 좋아요 · 댓글 · 구독 · 알림설정 부탁드립니다. 감사합니다.

Replit

핵심 가이드

코딩 없이 말로 만드는 나만의 홈페이지와 앱

2026년 2월 1 라이브 코딩 입문

프롬프트 작성 · 디자인 수정 · 배포 · 도메인 연결 · AI 모델 활용

목차

Part 1. Replit 시작하기

- › Replit이란?
- › 가입 및 플랜
- › 첫 프로젝트 만들기
- › 홈페이지 콘텐츠 준비하기
- › 프롬프트 작성의 기본

Part 2. 실전 홈페이지 만들기

- › AI에게 말로 설명하기
- › 디자인 수정하기
- › 이미지와 미디어 추가
- › Publish로 배포하기
- › 도메인 연결하기
- › 수정과 재배포

Part 3. 고급 기능 활용

- › 모바일 앱으로 만들기
- › AI Integrations (300+ AI 모델)
- › Connectors (외부 서비스 연결)
- › Figma Import & Visual Editor
- › 효과적인 프롬프팅 전략

부록

- › 자주 묻는 질문 (FAQ)
- › 오늘 당장 실행할 액션 아이템
- › 유용한 리소스

PART 1

Replit 시작하기

가입부터 첫 프로젝트 생성까지, 바이브 코딩의 첫걸음

Replit이란?

Replit은 자연어(일상 언어)로 앱을 만들 수 있는 온라인 플랫폼입니다. 코딩 경험이 전혀 없어도 내가 원하는 것을 말로 설명하면 AI가 홈페이지, 웹앱, 도구를 만들어줍니다.

기존 방식

홈페이지 외주 맡기기: 수백만 원
직접 코딩 배우기: 수개월 학습
서버/호스팅 설정: 기술 지식 필요
도메인 연결: 복잡한 설정

Replit 방식

말로 설명하면 AI가 만들어줌
코딩 지식 불필요
배포 버튼 하나로 공개
도메인 연결도 클릭 몇 번이면 끝

핵심: Replit은 만들기, 수정하기, 배포하기가 하나의 플랫폼 안에서 전부 해결됩니다. 별도의 서버, 호스팅, 배포 도구가 필요 없습니다.

가입 및 플랜

replit.com에 접속하여 Google, GitHub 등의 계정으로 간편하게 가입할 수 있습니다.

플랜	가격	주요 특징
Starter	체험 가능	기본 기능 이용, AI 프롬프트 제한 있음
Core	월 \$25 (연 \$20)	충분한 AI 사용량, 커스텀 도메인, 배포 포함
Core Pro	월 \$50 이상	대용량 프로젝트, 팀 협업, 우선 지원

Tip: 외주를 맡기면 홈페이지 하나에 수백만 원이 들지만, Replit Core 한 달이면 여러 개의 사이트를 만들 수 있습니다. 투자 대비 효과가 매우 높습니다.

첫 프로젝트 만들기

- 1 replit.com에 접속하여 로그인합니다. 별도 프로그램 설치의 필요 없습니다.
- 2 프로젝트 유형을 선택합니다. 홈페이지를 만들려면 웹앱(Web App)을 선택하세요.
- 3 프롬프트 입력란에 만들고 싶은 것을 한국어로 자세히 설명합니다.

4 **Start** 버튼을 누르면 AI가 자동으로 만들기 시작합니다.

포인트: 만들어지는 과정을 실시간으로 볼 수 있습니다. 완성되기 전에도 미리보기로 중간 결과를 확인할 수 있어요.

홈페이지 콘텐츠 준비하기

Replit에 프롬프트를 넣기 전에, 홈페이지에 들어갈 **콘텐츠를 미리 준비**하면 훨씬 좋은 결과물이 나옵니다. ChatGPT 같은 AI 챗봇에게 먼저 카피를 작성해달라고 요청하세요.

홈페이지 필수 섹션

섹션	내용	예시
히어로	한 줄 소개 + 부가 설명	"바이브 코딩으로 당신의 아이디어를 현실로"
서비스 소개	핵심 서비스 3가지	1:1 코칭, 그룹 강의, 템플릿 판매
실적/포트폴리오	숫자로 보여주는 성과	수강생 500+, 만족도 98%
고객 후기	실제 고객의 추천사	2~3개의 구체적인 후기
CTA	행동 유도 버튼	"지금 상담 신청하기"

비유: 콘텐츠 준비는 요리의 **재료 손질**과 같습니다. 재료가 잘 준비되어 있으면 AI 셰프가 훨씬 맛있는 요리를 만들어줍니다.

프롬프트 작성의 기본

Replit AI에게 잘 전달하려면 프롬프트를 **구체적이고 체계적으로** 작성해야 합니다.

나쁜 프롬프트

"멋진 홈페이지 만들어줘"

→ AI가 어떤 것을 만들지 판단하기 어려움

좋은 프롬프트

"1인 창업 코칭 서비스 퍼스널 브랜딩 홈페이지를 만들어줘. 히어로 섹션, 서비스 소개 3가지, 고객 후기, 상담 신청 CTA가 포함되어야 해. 다크모드를 지원하고 모던한 디자인으로 해줘."

좋은 프롬프트의 3요소

- 1 **목적:** 무엇을 만들 것인지 (퍼스널 브랜딩 홈페이지, SaaS 랜딩페이지 등)
- 2 **구성:** 어떤 섹션이 필요한지 (히어로, 서비스, 후기, CTA 등)
- 3 **스타일:** 디자인 방향 (모던, 미니멀, 다크모드 등)

PART 2

실전 홈페이지 만들기

AI와 대화하며 디자인 수정, 배포, 도메인 연결까지 한번에

AI에게 말로 설명하기

Replit의 AI 에이전트는 대화형으로 동작합니다. 처음에 전체 구조를 설명한 후, 세부 사항은 대화를 이어가며 수정하면 됩니다.

콘텐츠 준비



프롬프트 입력



AI가 생성



미리보기 확인



수정 요청

AI가 만드는 과정을 실시간으로 볼 수 있고, 진행 중에도 미리보기로 결과를 확인할 수 있습니다. 마치 옆에서 지켜보며 방향을 잡아주는 느낌입니다.

디자인 수정하기

완성된 결과물이 마음에 안 들면 그냥 **말로 수정을 요청**하면 됩니다. AI가 맥락을 이해하고 있기 때문에 "히어로 섹션 배경에 그라데이션 넣어줘"처럼 간단하게 요청해도 정확히 반영합니다.

자주 사용하는 수정 요청

요청	예시 프롬프트
색상 변경	"전체 톤을 파란색 계열로 바꿔줘"
레이아웃 변경	"서비스 섹션을 가로 3열로 배치해줘"
섹션 추가	"FAQ 섹션을 CTA 위에 추가해줘"
폰트/크기	"제목 폰트 크기를 더 키워줘"
다크모드	"다크모드와 라이트모드 모두 지원하게 해줘"
반응형	"모바일에서도 깔끔하게 보이게 해줘"

AI가 스스로 판단하기도 합니다: 배경색을 바꿨을 때 텍스트 색상이 안 어울리면 AI가 알아서 텍스트 색도 조정해줍니다.

이미지와 미디어 추가

프로필 사진이나 포트폴리오 이미지를 추가하고 싶다면 파일을 **드래그 앤 드롭**으로 올린 뒤, AI에게 "이 이미지를 프로필 사진으로 넣어줘"라고 요청하면 됩니다.

- 1 이미지 파일을 Replit 작업 공간으로 드래그 앤 드롭합니다

2 AI에게 이미지를 어디에 배치할지 말로 설명합니다

3 위치나 크기가 마음에 안 들면 다시 요청합니다

Tip: 이미지뿐 아니라 아이콘, 로고, 배경 이미지 등도 같은 방식으로 추가할 수 있습니다.

Publish로 배포하기

사이트가 마음에 들면 **Publish** 버튼 하나로 전 세계 누구나 접속 가능한 사이트로 공개할 수 있습니다.



배포 전 (미리보기)

나만 볼 수 있는 상태

다른 사람에게 주소를 알려줘도 접속 불가

배포 후 (Publish 완료)

고유한 웹 주소가 생성됨

전 세계 누구나 접속 가능

서브도메인: Publish 시 `네이름.replit.app` 형식의 주소가 자동으로 부여됩니다. 네이름 부분은 원하는 대로 설정할 수 있습니다.

도메인 연결하기

나만의 도메인(예: `mysite.com`)을 사용하고 싶다면 Replit 내에서 바로 연결할 수 있습니다.

1 **Replit에서 도메인 구매:** Replit 내에서 직접 도메인을 구매할 수 있습니다

2 **기존 도메인 연결:** 이미 가지고 있는 도메인을 DNS 설정으로 연결합니다

Tip: 처음에는 `네이름.replit.app` 주소로 시작하고, 나중에 사업이 커지면 커스텀 도메인을 연결해도 됩니다.

수정과 재배포

배포한 사이트를 수정해야 할 때는 Replit에서 AI에게 수정을 요청한 뒤 **Republish** 버튼을 누르면 됩니다. 기존 도구들처럼 복잡한 과정이 필요 없습니다.



PART 3

고급 기능 활용

모바일 앱, AI 모델, 외부 서비스 연동으로 가능성을 확장하는 법

모바일 앱으로 만들기

Replit은 **Apple App Store**와 **Google Play Store**에서 모바일 앱을 제공하는 유일한 바이브 코딩 플랫폼입니다. 스마트폰에서 앱을 설치하면 이동 중에도 프로젝트를 만들고 수정할 수 있습니다.

이런 상황에서 유용합니다

- 출퇴근 시간에 아이디어를 바로 구현
- 외출 중 급한 사이트 수정
- 카페에서 가볍게 프로젝트 시작
- 클라이언트 미팅 중 즉석 시연

모바일에서 할 수 있는 것

- 새 프로젝트 생성
- AI에게 수정 요청
- 실시간 미리보기
- Publish / Republish

내 주머니 안의 앱 빌더: PC가 없어도 스마트폰 하나로 아이디어를 현실로 만들 수 있습니다.

AI Integrations (300+ AI 모델)

Replit에는 **300개 이상**의 **AI 모델**이 내장되어 있습니다. 별도의 API 키나 계정 설정 없이 바로 활용할 수 있습니다.

AI 모델 제공사	활용 분야
OpenAI	텍스트 생성, 대화형 기능
Anthropic (Claude)	분석, 요약, 콘텐츠 작성
Google (Gemini)	텍스트, 이미지, 오디오, 비디오 생성
OpenRouter	다양한 오픈소스 모델 접근

활용 아이디어: "AI 고객 상담 챗봇을 만들어줘" 한마디면 Replit이 적절한 AI 모델을 연결해서 바로 동작하는 챗봇을 만들어줍니다. 복잡한 설정 과정이 전혀 없습니다.

Connectors (외부 서비스 연결)

Connectors는 내가 이미 사용하고 있는 서비스들을 Replit 앱에 쉽게 연결해주는 기능입니다. **로그인만 하면 연결 완료**이며, 복잡한 설정이 필요 없습니다.

연결 가능 서비스	활용 예시
Notion	Notion 데이터를 가져와 대시보드 생성
Google Drive	파일을 자동으로 정리하는 도구
Slack	알림 봇, 자동 메시지 전송
Dropbox	파일 관리 자동화 앱
HubSpot	고객 관리(CRM) 도구
Linear	프로젝트 관리 대시보드

예시: "Dropbox에 있는 파일을 자동으로 분류하는 앱을 만들어줘"라고 하면, Replit이 Dropbox 연결을 설정하고 앱을 만들어줍니다. 예전에는 전문가에게 맡겨야 했던 작업이 이제 말 한마디로 됩니다.

Figma Import & Visual Editor

디자인을 먼저 Figma에서 만든 뒤 Replit으로 가져올 수도 있습니다. **Figma Import** 기능으로 디자인 파일을 그대로 실제 동작하는 앱으로 변환할 수 있습니다.

Visual Editor

코드를 직접 보지 않고 시각적으로 편집할 수 있는 도구입니다. 드래그 앤 드롭으로 요소를 이동하고, 클릭으로 색상이나 텍스트를 변경할 수 있습니다.



효과적인 프롬프팅 전략

Replit AI를 200% 활용하기 위한 프롬프팅 전략을 정리했습니다.

#	전략	설명
1	한 번에 하나씩	여러 수정사항을 한꺼번에 요청하지 말고, 하나씩 요청하고 결과를 확인한 뒤 다음 수정으로 넘어가세요
2	구체적으로 설명	"이쁘게 해줘" 보다 "파란색 그라데이션 배경에 흰색 텍스트, 둥근 버튼으로 바꿔줘"가 훨씬 좋습니다
3	참고 자료 활용	좋아하는 웹사이트 스크린샷을 올리고 "이 느낌으로 만들어줘"라고 하면 효과적입니다
4	역할 부여	"너는 10년차 웹 디자이너야" 같은 역할을 부여하면 더 전문적인 결과물이 나옵니다
5	결과 확인 후 피드백	AI가 만든 결과를 보고 "여기는 좋은데, 이 부분은 이렇게 바꿔줘"처럼 구체적으로 피드백하세요

주의: 프롬프트가 너무 길면 AI가 핵심을 놓칠 수 있습니다. **핵심 요구사항을 먼저 전달**하고, 세부사항은 이후 대화에서 추가하는 것이 좋습니다.

부록: 자주 묻는 질문 (FAQ)

질문	답변
코딩을 전혀 몰라도 되나요?	네, 전혀 몰라도 됩니다. 한국어로 원하는 것을 설명하면 AI가 만들어줍니다.
만든 사이트의 소유권은 누구에게 있나요?	Replit에서 만든 모든 것은 사용자의 소유입니다.
내 도메인을 연결할 수 있나요?	네, Core 플랜 이상에서 커스텀 도메인을 연결할 수 있습니다.
모바일에서도 만들 수 있나요?	네, iOS/Android 앱에서 동일한 기능을 사용할 수 있습니다.
홈페이지 말고 다른 것도 만들 수 있나요?	네, 웹앱, 자동화 도구, AI 챗봇, 대시보드, SaaS 등 다양한 것을 만들 수 있습니다.
만든 앱의 보안은 안전한가요?	Replit은 보안 환경에서 앱을 구동하므로 안전합니다. 다만 결제/개인정보를 다루는 앱은 추가 보안 검토를 권장합니다.

부록: 오늘 당장 실행할 액션 아이템

- 1 Replit 가입하기:** replit.com에 접속하여 계정을 만드세요. 이메일이나 Google 계정으로 간편하게 가입할 수 있습니다.
- 2 콘텐츠 준비하기:** 내 홈페이지에 들어갈 소개, 서비스, 후기 내용을 정리하세요. AI 챗봇을 활용하면 더 빠릅니다.
- 3 첫 홈페이지 만들어보기:** 준비한 콘텐츠를 Replit에 붙여넣고 Start를 누르세요. 몇 분이면 내 홈페이지가 완성됩니다.

부록: 유용한 리소스

리소스	설명
Replit 갤러리	replit.com/gallery에서 다른 사람들이 만든 앱을 구경하고 영감을 얻으세요
프롬프팅 가이드	Replit 공식 문서의 효율적인 프롬프팅 가이드를 참고하세요
모바일 앱	replit.com/mobile에서 모바일 앱을 다운로드하세요

Replit 핵심 가이드 | 2026년 2월 기준

Replit은 빠르게 발전하고 있습니다. 최신 기능은 replit.com에서 확인하세요.